



Machine Learning 의 기초

1. Machine Learning 의 정의

- 명시적인 프로그래밍 없이 컴퓨터가 학습하는 능력을 갖추게 하는 연구 분야이다. (아서 새뮤얼, 1959)
- 어떤 작업 T 에 대한 컴퓨터 프로그램의 성능을 P 로 측정 했을 때 경험 E 로 인해 성능이 향상 되었다면, 이 컴퓨터 프로그램은 작업 T 와 성능 측정 P 에 대해 경험 E 로 학습한 것이다.

➡ 어떤 정보들을 컴퓨터에게 입력해 주면,
특정 알고리즘에 의해 **패턴을 파악**하고 **다른 사례에 대응** 하게 된다.

2. Machine Learning 사용 사례

- 스팸으로 분류되는 단어나 구절을 입력 한다.
- 해당 단어나 구절이 나올 경우 자동으로 스팸 처리가 된다.
- 일반적인 프로그램의 경우 일전에 없던 스팸 구절이 나타날 경우 판단할 수 없게 된다.(if 구문을 추가)
- Machine Learning 에서는 그동안 파악했던 패턴을 통해 스팸 여부를 구분해 낸다.

사례
제품의 이미지 들을 분석해 생산 라인의 제품을 분류 한다.
정상적인 뇌와 종양이 있는 뇌 스캔 사진 들을 분석하여 종양을 진단 한다.
뉴스기사 들을 분석하여 뉴스 기사를 분류 한다.
자연어 명령어 들을 분석하여 특정 명령에 해당 서비스 를 연결 한다.
신용카드 사용 패턴 들을 분석하여 부정사용을 탐지 한다.
소비자들의 구매패턴 들을 분석하여 각 군집(나이/성별/직업) 별 마케팅 전략을 세운다.

3. Machine Learning 시스템의 종류

종류	설명
지도 학습	훈련 데이터에 레이블이라는 원하는 답을 함께 입력(문제와 답을 통해 패턴을 형성)
비지도 학습	훈련 데이터에 레이블을 제공 하지 않음(비슷한 것끼리 알아서 분류)
준 지도 학습	비지도 학습으로 분류 후, 분류 군집에 레이블을 다는 형태
강화 학습	환경을 관찰해서 행동을 하고 결과에 따라 보상과 벌점으로 최상의 전략을 찾는 방법
배치 학습	가용 데이터를 한번에 사용하여 학습(새로운 데이터가 생길 경우 처음부터 다시 학습)
온라인 학습	미니 배치를 통해 연속되는 입력 데이터를 점진적으로 학습
사례 기반 학습	입력된 데이터를 훈련 데이터 와 유사 도를 측정하여 분류
모델 학습	샘플의 경향성을 특정 모델로 만들고 이를 통해 입력되는 데이터를 분류

4. Machine Learning 에서 중요한 것은 바로 training data

- 사람도 기계도 올바른 내용으로 올바르게 배워야만 정확도 높은 결과를 낼 수 있다.

머신 러닝	시험 준비
충분치 않은 양의 훈련 데이터	적은 양의 기출 문제
대표성 없는 훈련 데이터	특정 과목에만 편향된 기출 문제
낮은 품질의 데이터	변별력이 없는 기출 문제
관련 없는 특성	시험 범위에 벗어난 기출 문제

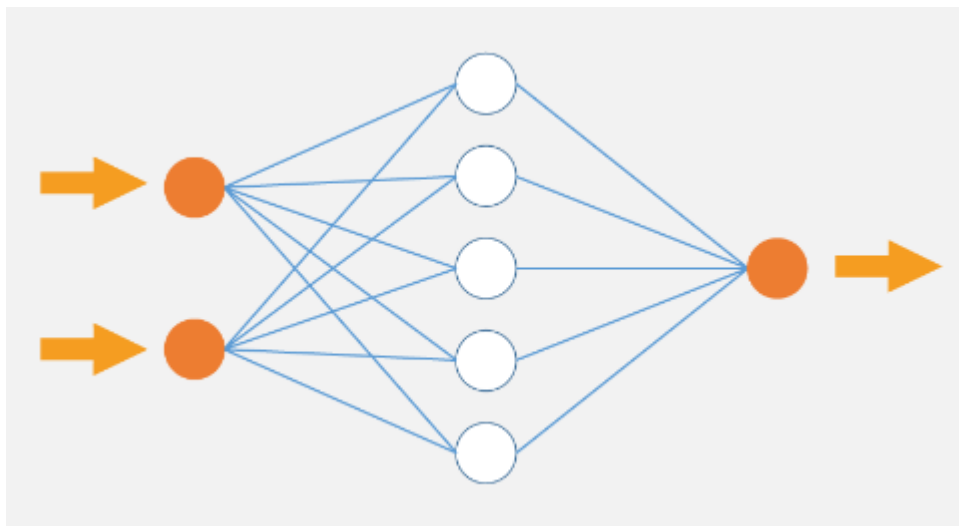


SCIKIT LEARN

Scikit learn 이란?

- Scikit learn 은 python의 대표적인 **machine learning** 라이브러리이다.
- 주로 분류 쪽에 특화되어 있다.

종류	설명
machine learning	제공 받은 데이터로 학습하고 패턴을 찾아내 입력 값과 정답 사이의 연관성을 찾아 추론한다.
deep learning	machine learning 의 일부로써, 인공 신경망 학습 이나 강화 학습을 사용 한다.

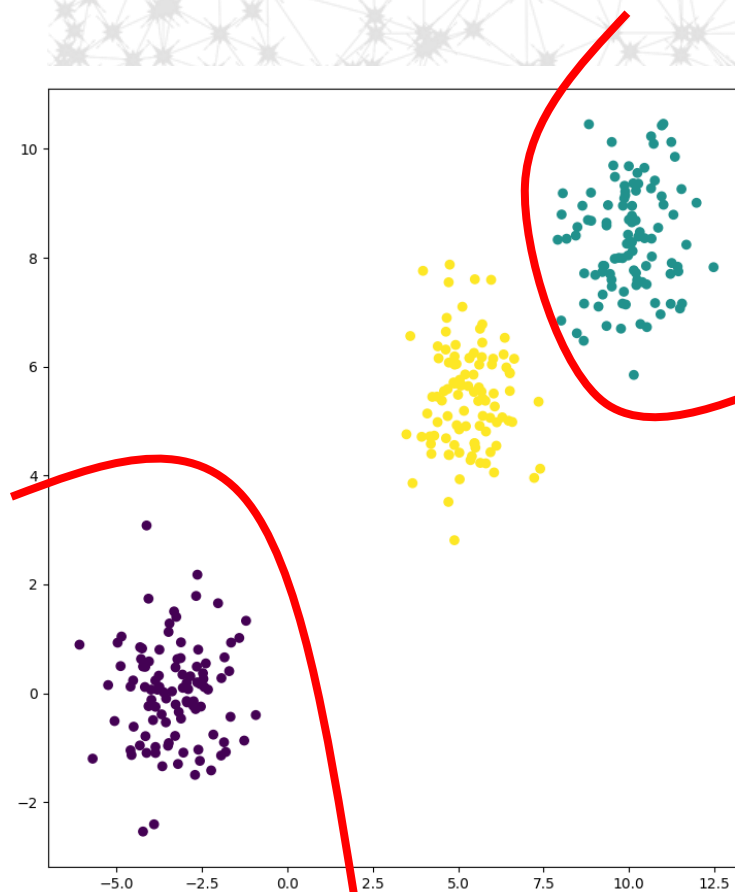
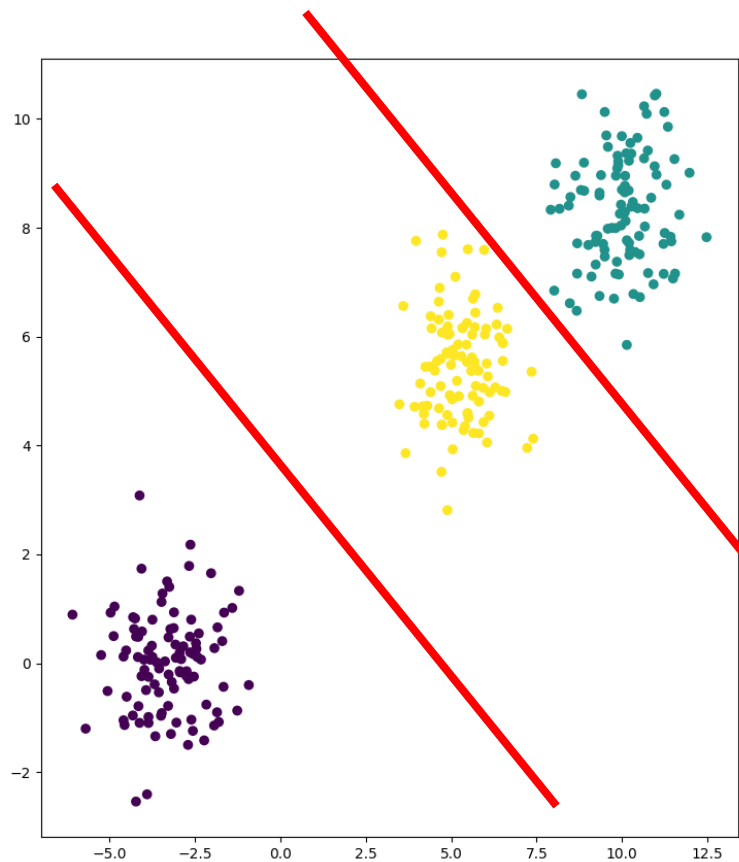


Machine learning

deep learning

machine learn 의 본질

- Machine learning 은 결과 적으로 특정한 데이터가 있을 때 그 결과를 분류 하는 데에 있다.
- 그리고 좀더 세밀한 구분을 하기 위한 여러가지 시도를 수행하게 된다.



독립변수(X) 와 종속변수(y)

- 독립 변수는 연구자가 의도적으로 값을 변경 할 수 있는 변수 이다.
- 종속 변수는 독립 변수에 따라 종속적으로 변화되는 변수 이다.

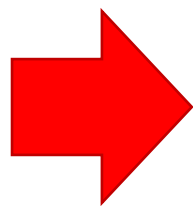
X

3

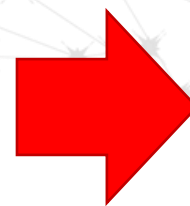
2

5

- **y** 는 x 값 변화에 따라 변화 하므로 X 에 **종속**되어 있다.



function



y

7

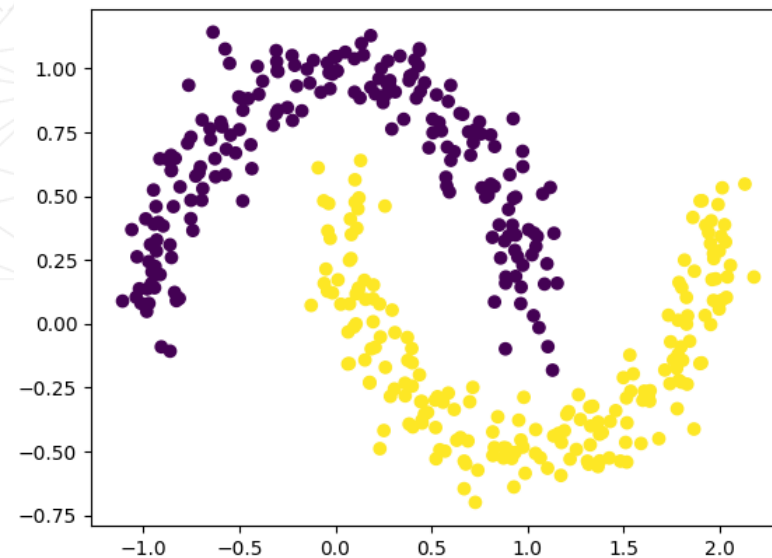
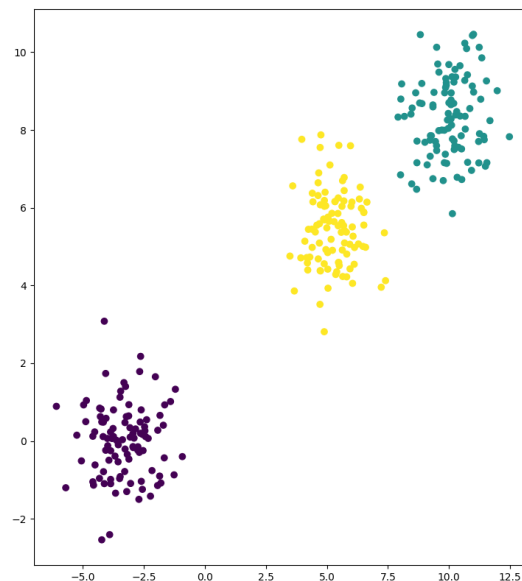
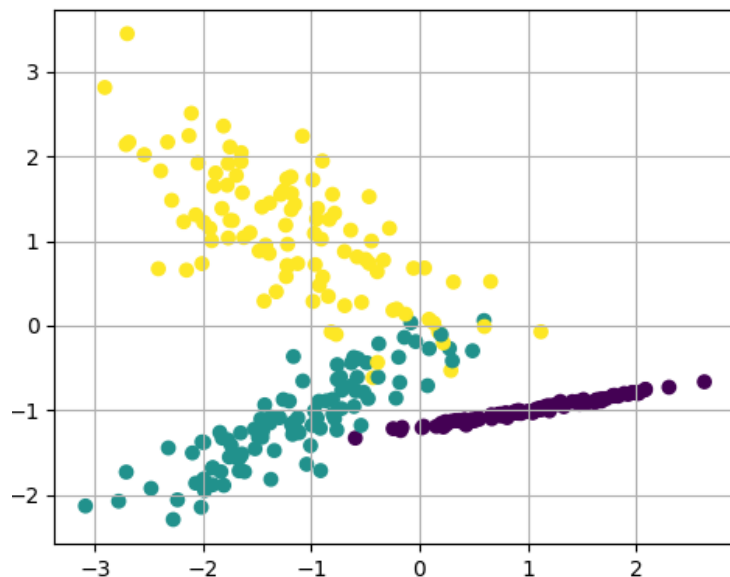
8

5

분류 Data 생성

- Scikit learn 에서는 machine learning 에서 사용할 기본적인 데이터를 제공 해 준다.

함수	설명
<code>make_classification()</code>	분류(classification) 모형 테스트를 위한 데이터 생성
<code>make_blobs()</code>	등방성 가우시안 정규분포를 이용해 테스트 데이터 생성
<code>moons()</code>	초승달 모양 클러스터 두개의 테스트 데이터를 생성



분류 Data 생성 - make_classification

- Make_classification 함수를 이용해 분류 데이터를 생성해 보자

```
x,y=make_classification(n_features=20,n_classes=2,n_informative=2,n_clusters_per_class=1,  
n_redundant=0,n_samples=500)
```

실습파일 : 01_classification.ipynb

옵션	설명	기본값
n_features	입력 값에 포함되어야 할 전체 특성(분류기준)의 수	20
n_classes	분류되어야 할 class 의 수(예: 스팸/정상, 합격/불합격)	2
n_informative	전체 입력 값 중에 실제로 구분하는데 유용한 정보를 가진 특성의 수	2
n_clusters_per_class	하나의 클래스내에 데이터가 뭉쳐있는 군집의 수	2
n_redundant	유용한 정보를 가진 특성들을 선형 조합하여 만든 중복된 특성의 수 예: '키(cm)'라는 유용한 특성이 있을 때, '키(inch)'라는 특성을 추가하면 이는 중복된 특성이 된다.	2
n_samples	보여줄 샘플 개수	-

분류 Data 생성 - make_classification

- `n_classes` 를 3으로 지정하여 실행시키면 아래와 같은 오류가 나타난다.

```
x, y = make_classification(n_features=2, n_classes=3, n_clusters_per_class=2,  
                           n_informative=2, n_redundant=0, n_samples=500)
```

ValueError: `n_classes(3) * n_clusters_per_class(2)` must be smaller or equal `2**n_informative(2)=4`

분류(`n_classes`) 3개, 각 분류별 포인트(`n_clusters_per_class`) 2개
이면 총 6개의 분류 포인트가 필요하다.

분류 포인트는 주요 정보(`n_informative`) 변수의 조합으로
만들어지는데 2개 밖에 없으므로 어떤 조합을 하더라도 4개
이상 나오지 못한다.

분류 Data 생성 - make_classification

- 3개의 분류 내 2개의 분류 포인트를 설정하면 총 6개의 분류 포인트가 필요
- 하지만 주요 정보는 2개 이므로 분류포인트는 4개 밖에 만들 수 없음



해결방법 1. $n_classes$ 를 줄인다. ($2 \times 2 \leq 2 \times 2$)

$n_classes=2$ $n_clusters_per_class=2$ $n_informative=2$

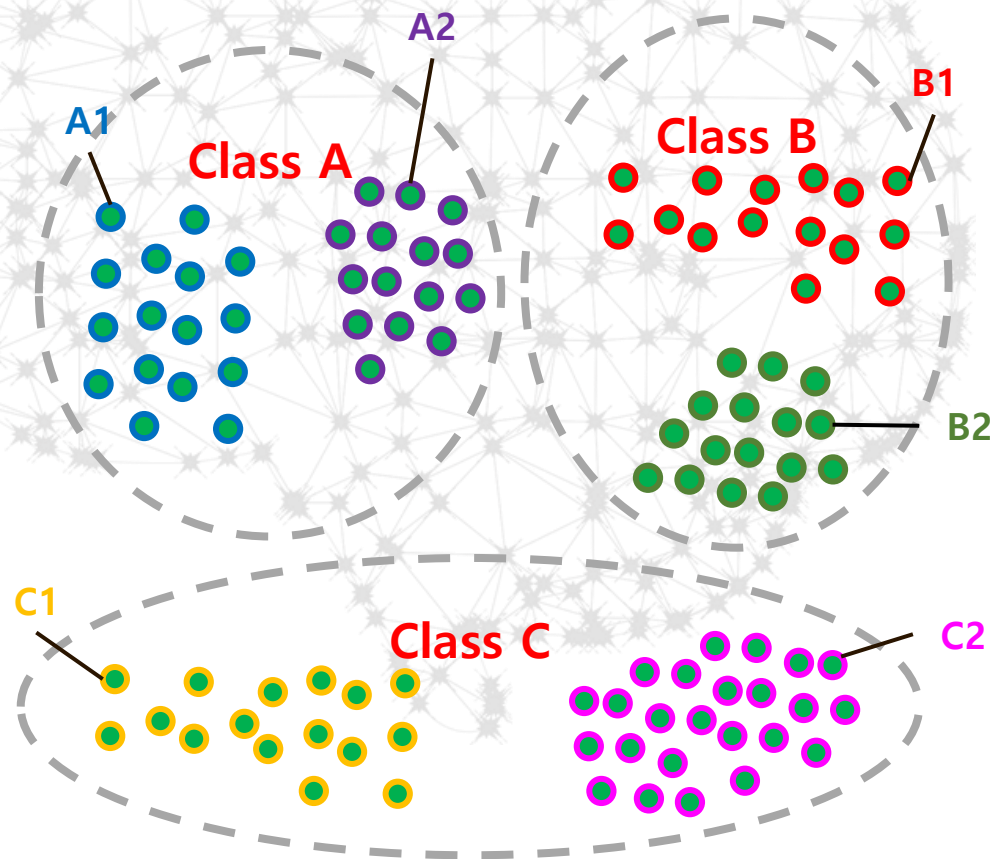
해결방법 2. $n_clusters_per_class$ 를 줄인다. ($3 \times 1 \leq 2 \times 2$)

$n_classes=3$ $n_clusters_per_class=1$ $n_informative=2$

해결방법 3. $n_informative$ 를 늘린다. ($2 \times 2 \leq 2 \times 3$)

$n_classes=2$ $n_clusters_per_class=2$ $n_informative=3$

$n_classes=3$
 $n_clusters_per_class=2$
 $n_informative=2$



분류 Data 생성 - make_classification

- 해결방법 3 에서도 오류가 발생?
- `n_informative` 은 입력 값 중에 주요 정보를 추출 한다.
- 그러므로 `n_features` 는 `n_informative` 보다 작으면 안된다.
- 아래 수치 관계를 지켜야 한다.

```
n_classes * n_clusters_per_class <= 2 ** n_informative  
n_features >= n_informative + n_redundant + n_repeated
```

`n_features=2`

`n_classes=2`

`n_clusters_per_class=2`

`n_informative=3`

분류 Data 생성 - make_blobs

- make_blobs 는 등방성 가우시안 정규분포 를 이용해 데이터 생성한다.
- 등방성은 같은 방향으로 나가는 성질을 의미한다.

```
x, y = make_blobs(n_samples=300, n_features=2, centers=2, cluster_std=1)
```

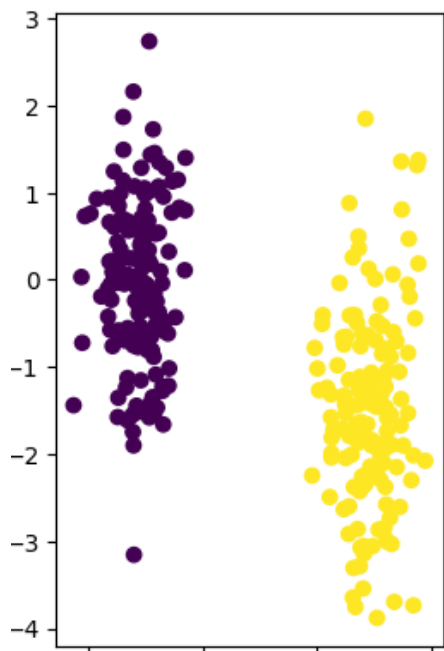
실습파일 : 02_blobs.ipynb

옵션	설명	기본값
n_samples	보여줄 샘플 개수	100
n_features	입력한 독립변수	2
centers	결과로 나타날 군집 수	3
cluster_std	클러스터의 표준 편차. 값이 클수록 데이터가 넓게 퍼짐	1.0

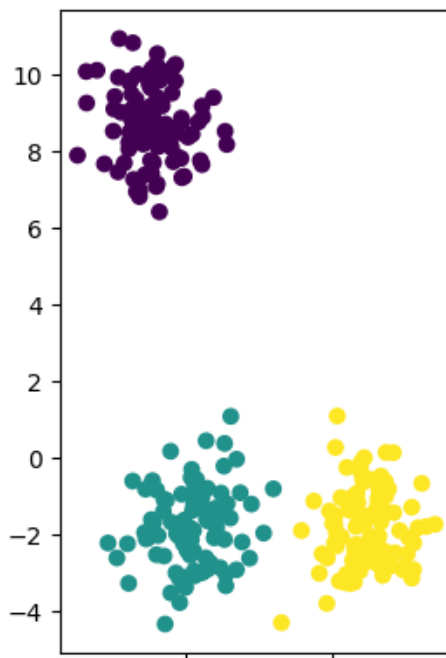
분류 Data 생성 - make_blobs

- 각 항목을 조정하면서 어떤 변화가 있는지 살펴보자

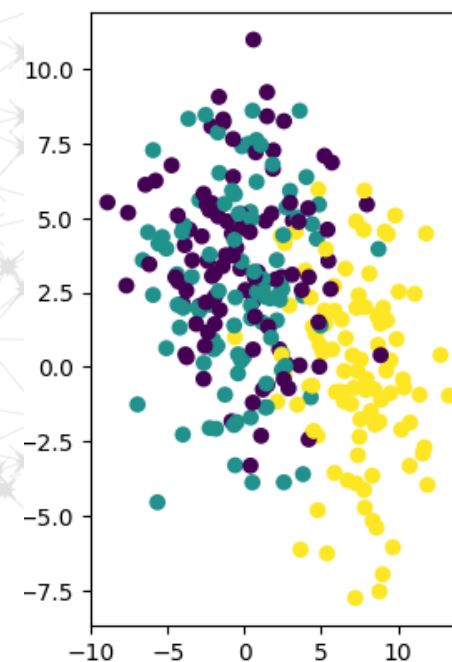
n_features=2,
centers=2,
cluster_std=1



n_features= 2 ,
centers=3,
cluster_std=1



n_features= 2 ,
centers=3,
cluster_std=3



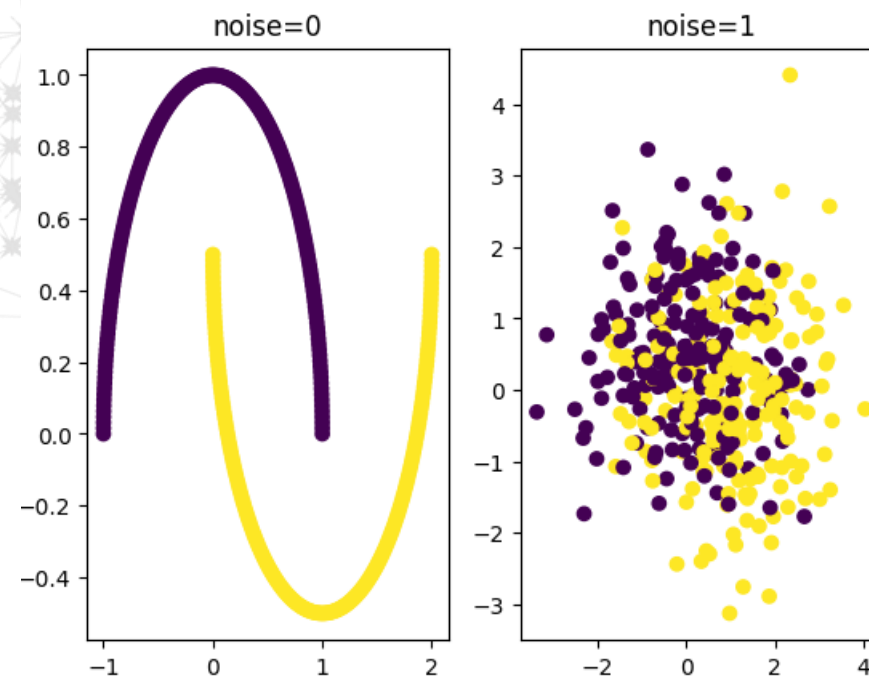
분류 Data 생성 - moons

- `make_moons` 는 초승달 모양의 클러스터 데이터를 생성 한다.
- 직선을 활용해서 분류할 수 없는 데이터를 만들 때 활용한다.

```
x,y=make_moons(n_samples=400, noise=0.2)
```

실습파일 : 03_moons.ipynb

옵션	설명	기본값
<code>n_samples</code>	보여줄 샘플 개수	100
<code>noise</code>	초승달 모양에서 퍼지는 정도(0~1)	-



간단한 Machine Learning 수행

- Scikit learn 에서 제공 해 주는 붓꽃(iris) 데이터를 불러와 데이터 시각화 및 학습, 테스트 까지 진행 해 보자

단계	설명
1. 데이터 불러오기	사용할 데이터를 불러온다.
2. Data set 분할	학습 데이터와 학습 완료 후 테스트 할 데이터를 나눈다(70:30 정도)
3. Data 시각화	데이터의 상태를 시각적으로 확인해 본다.
4. 학습 및 시험	학습 알고리즘을 실행하여 학습을 시키고 정확도를 시험 해 본다.

1단계. Data 불러오기

- 우선 데이터를 가져와 어떤 기준으로 무엇을 할 것인가에 대한 정의가 먼저 필요 하다.
- 먼저 불러올 데이터와 목표를 살펴보자

목적

- 붓꽃(iris)의 데이터 중 꽃잎(petal)과 꽃받침(sepal)의 폭과 길이를 통해 품종을 구분하려고 한다.

사용할 데이터	설명
data	붓꽃의 꽃잎(petal)과 꽃받침(sepal)의 폭과 길이 데이터
target	품종 정보[0, 1, 2]
target_name	품종 정보에 대응하는 이름['setosa', 'versicolor', 'virginica']
feature_name	Data 정보에 대응하는 속성 이름['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

```
from sklearn.datasets import load_iris

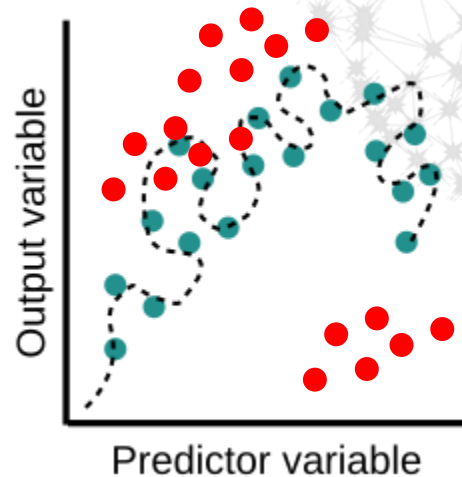
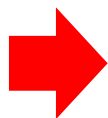
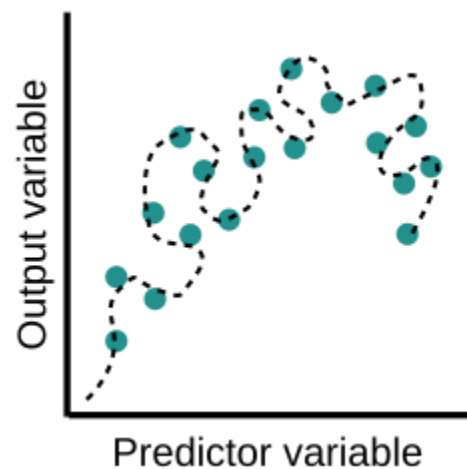
iris = load_iris()
print(f'iris 의 키 : {iris.keys()}')
print(f'입력되는 속성의 종류 : {iris.feature_names}')
print(f'맞춰야할 답의 종류 : {iris.target_names}')
```

실습파일 : 04_first_ml.ipynb

2단계. Data 분할

- 만약 연습문제로 나온 100 문제가 그대로 시험에 나온다면 너무 높은 점수가 나올 것이다.
- 이것을 **overfitting** 이라고 부른다.
- 그래서 받아온 데이터의 70% 정도만 학습에 활용하고 나머지는 테스트에 활용하도록 한다.

• 과 적합 예시



2단계. Data 분할

- 데이터를 불러와 분제와 답을 나누고, 연습문제와 실전 문제로 분리해 보자

```
# data: 입력데이터
x = iris.data
# target : 정답 데이터
y = iris.target
print(x[:10])
print(y[:10])
```

```
# 2. 데이터 분할
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.3,
    random_state=34,
    shuffle=True)
```

옵션	설명
x	입력 데이터
y	정답 데이터
test_size	데이터 분리 시 시험용으로 얼마나 분배할 것인가
random_state	섞을 때 seed 값
shuffle	분할 전 섞어줄지 여부

3단계. 시각화(Data visualization)

- 데이터 수치만 가지고는 데이터를 한눈에 파악하기 쉽지 않다.
- 그래서 시각화를 통해 데이터 분포 등을 파악한다.
- 꽃받침(sepal) 과 꽃잎(petal) 의 크기로 품종(color 로 구분 할 예정) 이 잘 구분되어 지는가?

```
import matplotlib.pyplot as plt
import pandas as pd
df = pd.DataFrame(x_train, columns = iris.feature_names)

plt.subplot(121)
plt.scatter(df['sepal length (cm)'], df['sepal width (cm)'], c=y_train, cmap=plt.cm.rainbow)
plt.xlabel('sepal length (cm)')
plt.ylabel('sepal width (cm)')

plt.subplot(122)
plt.scatter(df['petal length (cm)'], df['petal width (cm)'], c=y_train, cmap=plt.cm.rainbow)
plt.xlabel('petal length (cm)')
plt.ylabel('petal width (cm)')
plt.subplots_adjust(wspace=0.3)
plt.show()
```

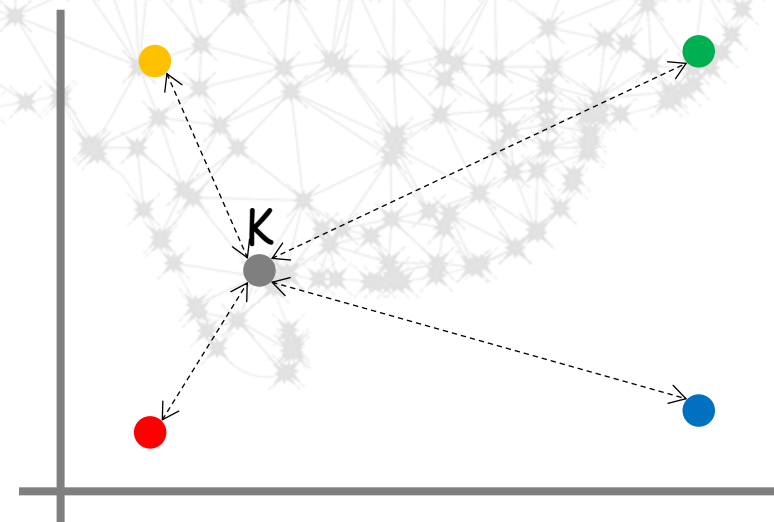
4 단계. 학습 및 시험

- Machine learning algorithm 은 scikit-learn 에서 제공해 주며 우리는 활용하기만하면 된다.
- 각 algorithm 의 원리를 알고 어느 부분에 활용해야 할지를 잘 결정 해야 한다.

사용할 알고리즘 : KNN(K Nearest Neighbor) - K 근접 알고리즘

* 어떤 값 K가 누구와 가까운 지에 따라 분류가 구분된다.

```
from sklearn.neighbors import KNeighborsClassifier  
class_knn = KNeighborsClassifier(n_neighbors=3) #알고리즘사용  
class_knn.fit(x_train, y_train) #학습  
pred_y = class_knn.predict(x_test) #예측 값 얻기
```



4 단계. 학습 및 시험

- 학습결과에 따른 예측 정확도를 평가 해 점수를 산출해 보자
- 점수는 보통 0.8 이상정도가 추천된다.

```
score=model.score(x_test, y_test)
```

5 단계. 저장

- 완성된 학습 모델은 파일로 저장하고 추후에 호출하여 사용하게 된다.

```
from joblib import dump
# 학습한 알고리즘을 저장하여 다른곳에서 사용할수도 있다.
root = '/content/drive/MyDrive/Data'
dump(class_knn, f'{root}/knn_iris.joblib')
```

score & cross_val_score

- 학습결과 점수산출을 해 주는 score 와 cross_val_score 를 비교해 보자

Score	Cross_val_score
Train data로 학습한 model 로 test data 의 예측 정확도 측정	데이터를 n 분할 후 각 분할 안에서 테스트를 하고 평균을 내는 방식
Train_test_split() 으로 데이터를 분할 해 줘야 한다.	Shuffle() 을 통해 데이터를 섞어 줘야 한다.
Test data 에 대해 어떤 예측을 하는지 추후 확인 해 볼 수 있다.	교차 검증 이므로 보다 합리적 이고 객관적이다.

```
score=model.score(x_test, y_test)
```

```
shuffle(x,y,random_state=33)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(x,y)
scores = cross_val_score(model,x,y)
```

실습파일 : 05_score.ipynb

저장된 학습 model 활용

- 이전에 저장했던 model 을 가져와 활용해 보자

```
from joblib import load
path = '/content/drive/MyDrive/Data/knn_iris.joblib'
names = ['Setosa', 'Virsicolor', 'Virginica']

class_knn = load(path)
sample = [[5, 5, 3, 2], [2, 4, 3, 5]]
results = class_knn.predict(sample)
```

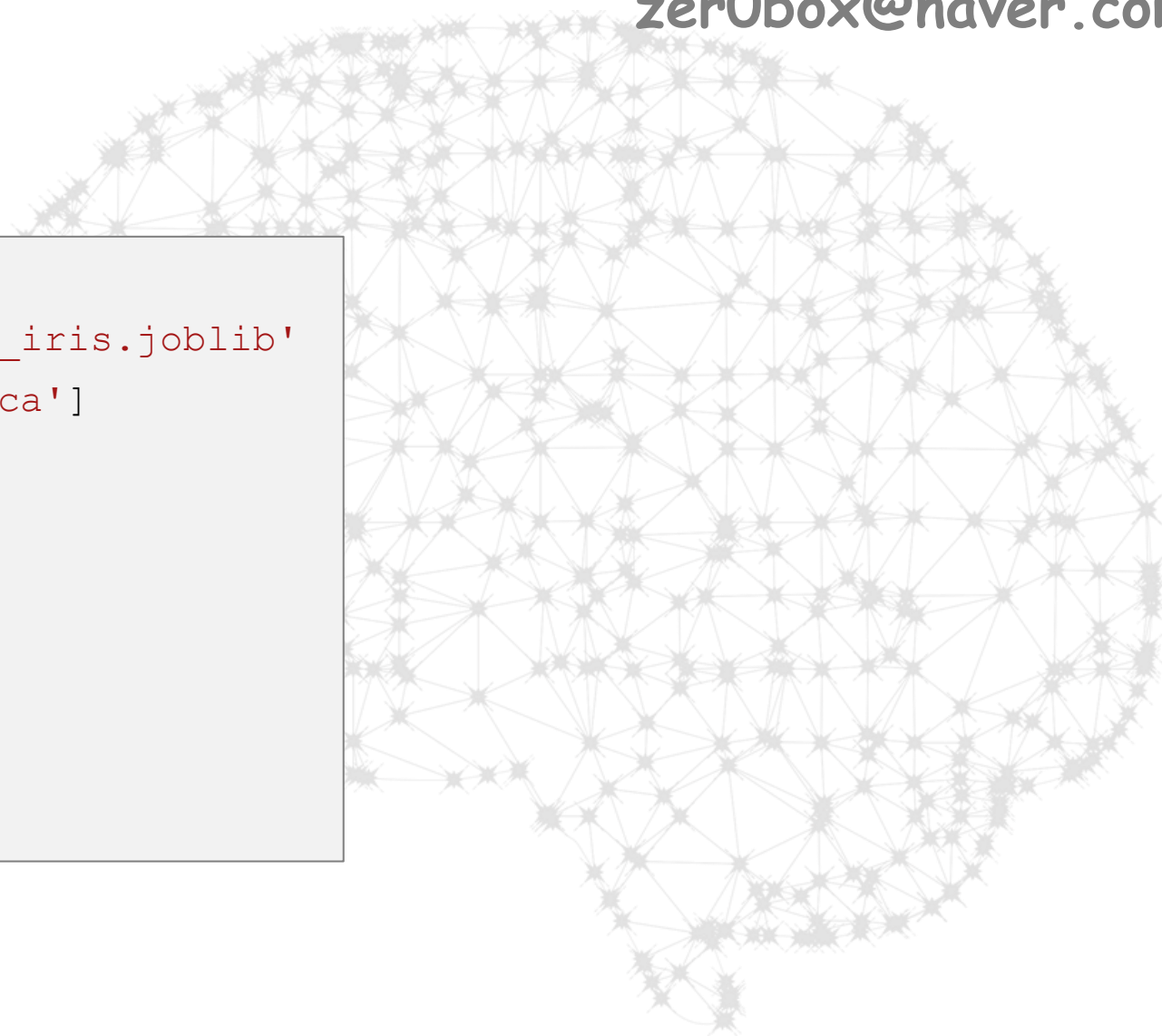
```
for r in results:
```

```
    print(names[r])
```



Setosa
Virginica

실습파일 : 06_load.ipynb

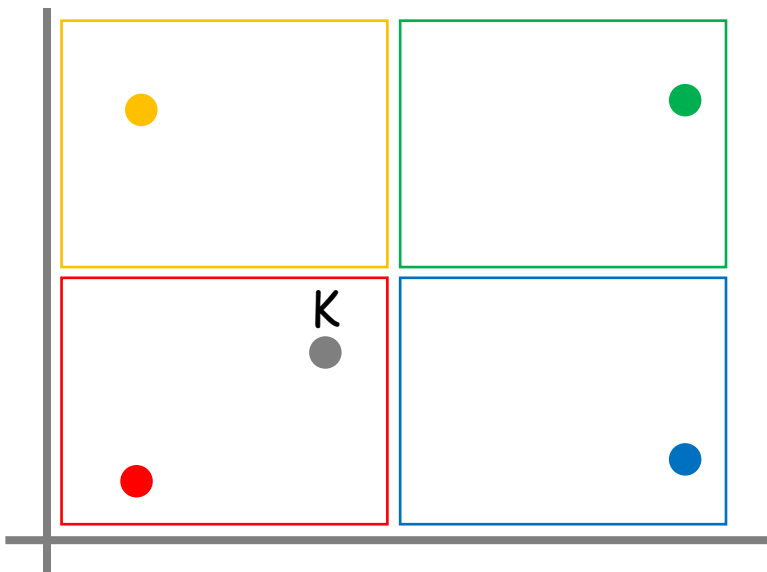




K Neighbor

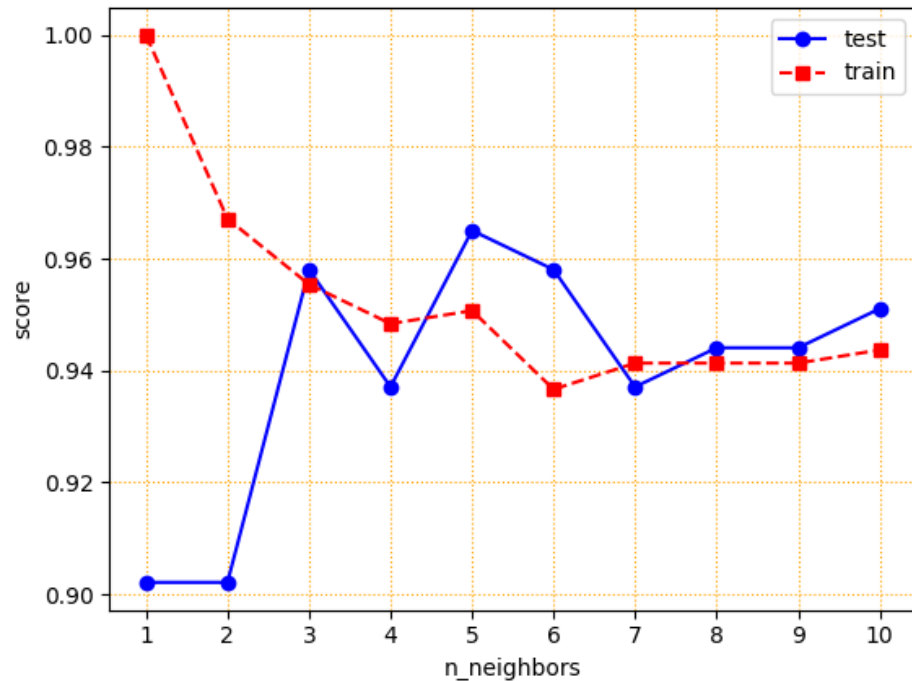
K Neighbors Classifier

- 앞에서 사용한 K 이웃 분류 알고리즘은 k 가 어느 이웃에 근접 한지에 따라 분류(classifier)되는 알고리즘 이다.
 - 분류(classifier)란 미리 정의된 class label 중 하나를 예측 하는 것이다.
- 예) 이 사람은 누구? 지금 온 메일은 SPAM 메일 일까?

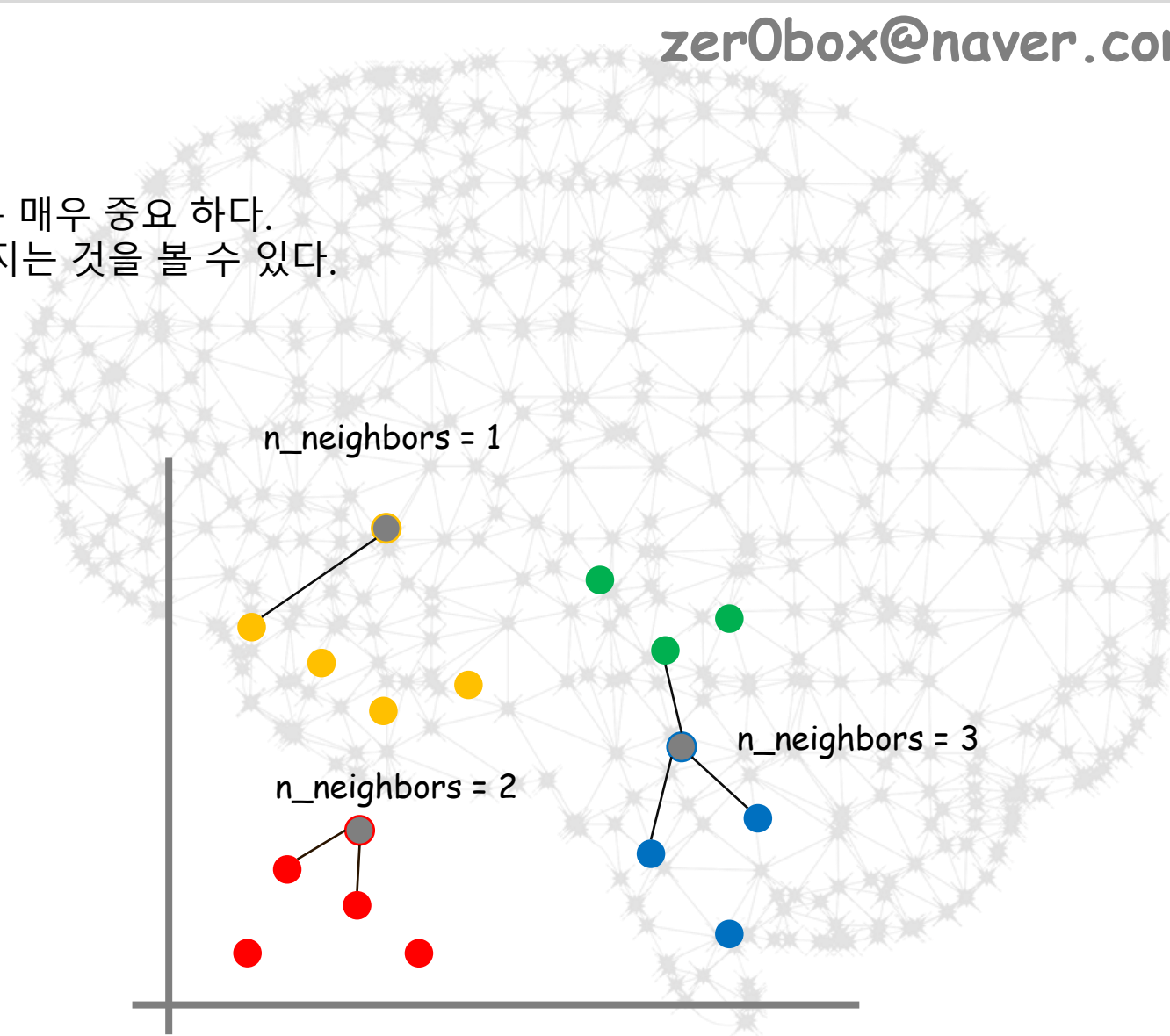


K Neighbors Classifier

- K Neighbors Classifier 에서 `n_neighbors` 값은 매우 중요 하다.
- `n_neighbors` 의 값에 따라 예측 정확도가 달라지는 것을 볼 수 있다.



실습파일 : 07_knn_class.ipynb

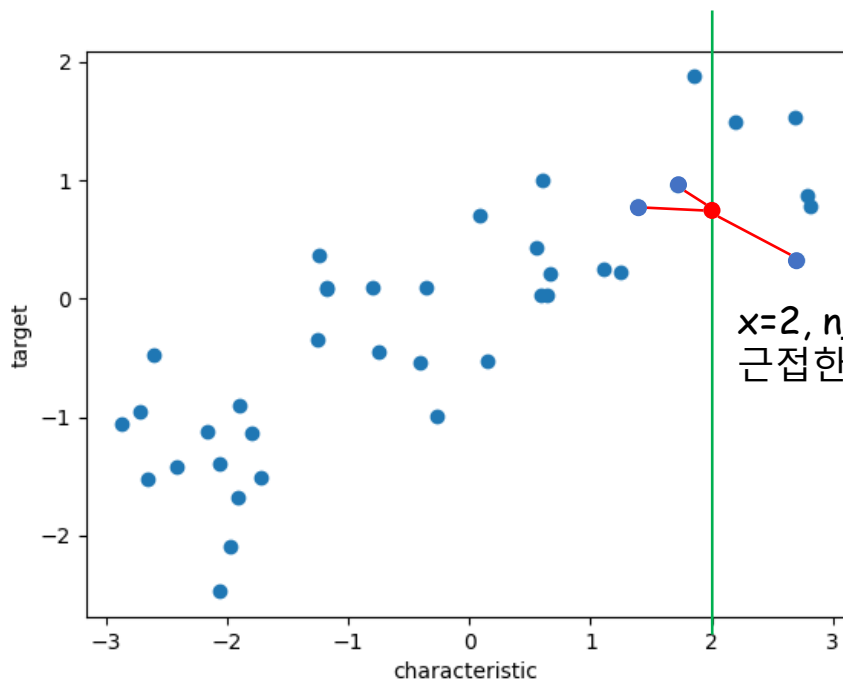


K Neighbors Regression

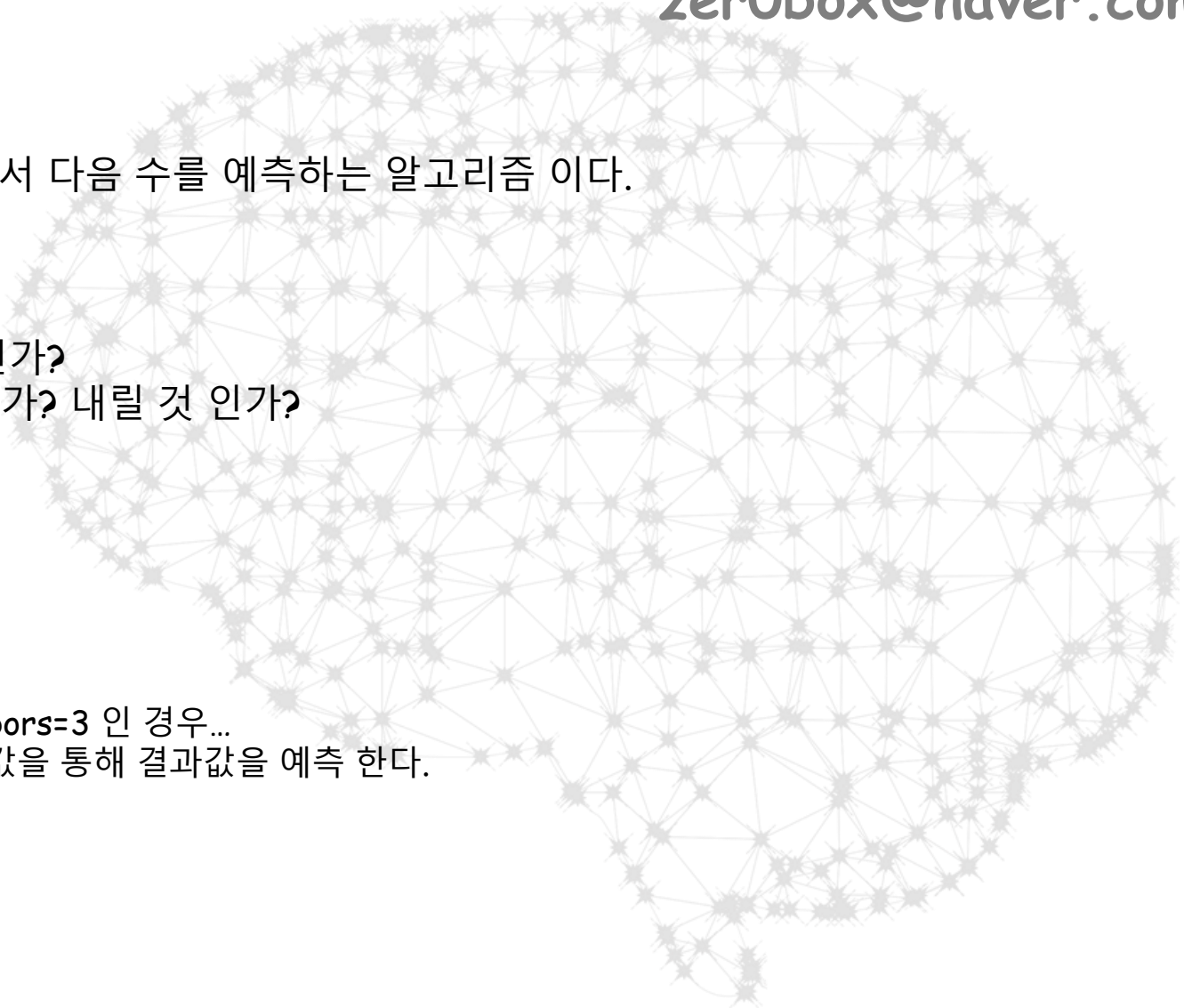
- 회귀(regression) 은 연속적인 숫자 데이터 속에서 다음 수를 예측하는 알고리즘 이다.
- 분류(classifier) 와는 분명한 차이가 있다.

* 회귀(regression) : 내일의 주가지수는 얼마 인가?

* 분류(classifier) : 내일 주가지수는 오를 것 인가? 내릴 것 인가?



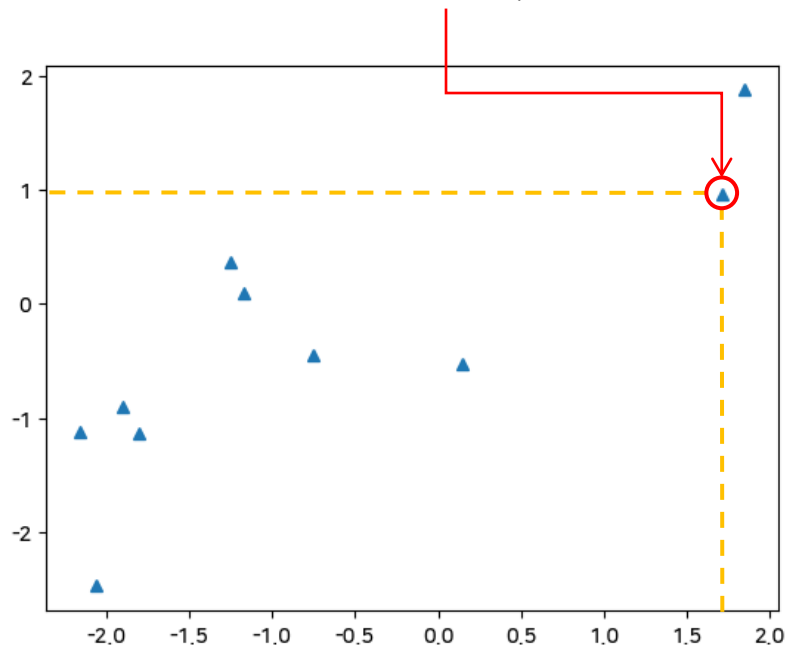
x=2, n_neighbors=3 인 경우...
근접한 3개의 값을 통해 결과값을 예측 한다.



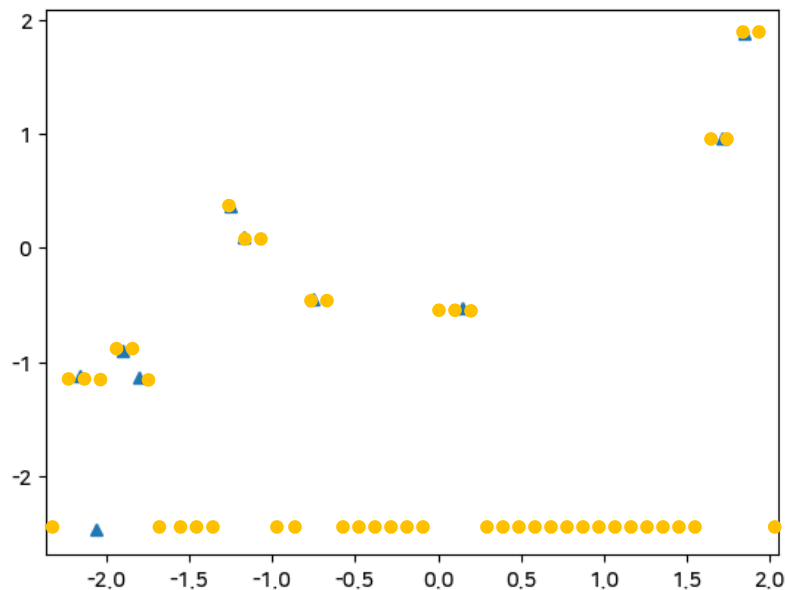
K Neighbors Regression

- `n_neighbors` 의 값에 따라 과적합이 어떻게 일어나는지 확인해 보자
- 이를 위해 학습데이터가 어떻게 학습 되었는지에 대한 내용을 확인해 보자

1. $\hat{y}$ 의 x 좌표는 입력 값이고, y좌표는 응답 값이다.



2. -3~3 까지의 값을 세밀하게 입력하여 y값을 예측하게 한다.



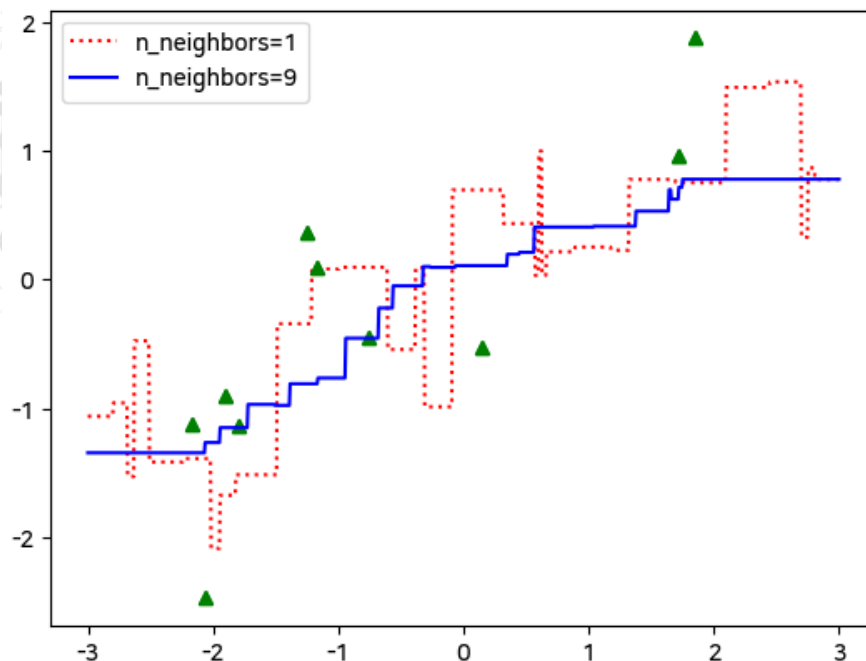
3. 세밀한 점들이 모이면 예측 곡선을 만들 수 있다.

K Neighbors Regression

- `n_neighbors=1` 인 경우 각 데이터가 주는 영향이 커서 **overfitting** 이 일어난다.
- `n_neighbors=9` 인 경우 너무 많은 주변 영향을 받다 보니 **under fitting** 이 일어난다.

```
import numpy as np
line = np.linspace(-3,3,1000).reshape(-1,1)
for n,c in zip([1,9],[':r','-b']):
    model = KNeighborsRegressor(n_neighbors=n).fit(x_train, y_train)
    plt.plot(x_test,y_test,'^g')
    result = model.predict(line)
    plt.plot(line, result,c,label=f'n_neighbors={n}')
plt.legend()
plt.show()
```

실습파일 : 08_knn_reg.ipynb



요약(Summary)

K Neighbors 의 장점

- 높은 정확도
- 모델이 단순하고 구현이 쉽다.

K Neighbors 의 단점

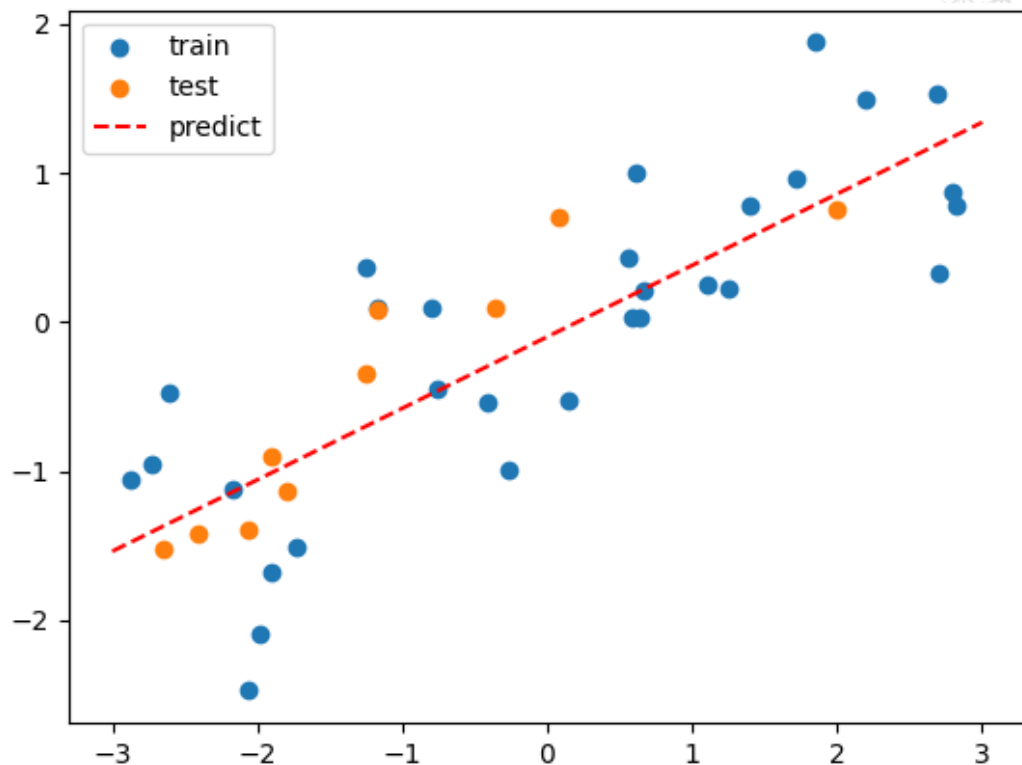
- Training data set 의 수가 많으면 예측이 느려진다.
- 많은 특성(입력되는 데이터 종류의 수) 을 갖은 data set에는 잘 동작하지 않는다.
- 희소한 data set(대부분이 0인 data) 과도 잘 맞지 않는다.



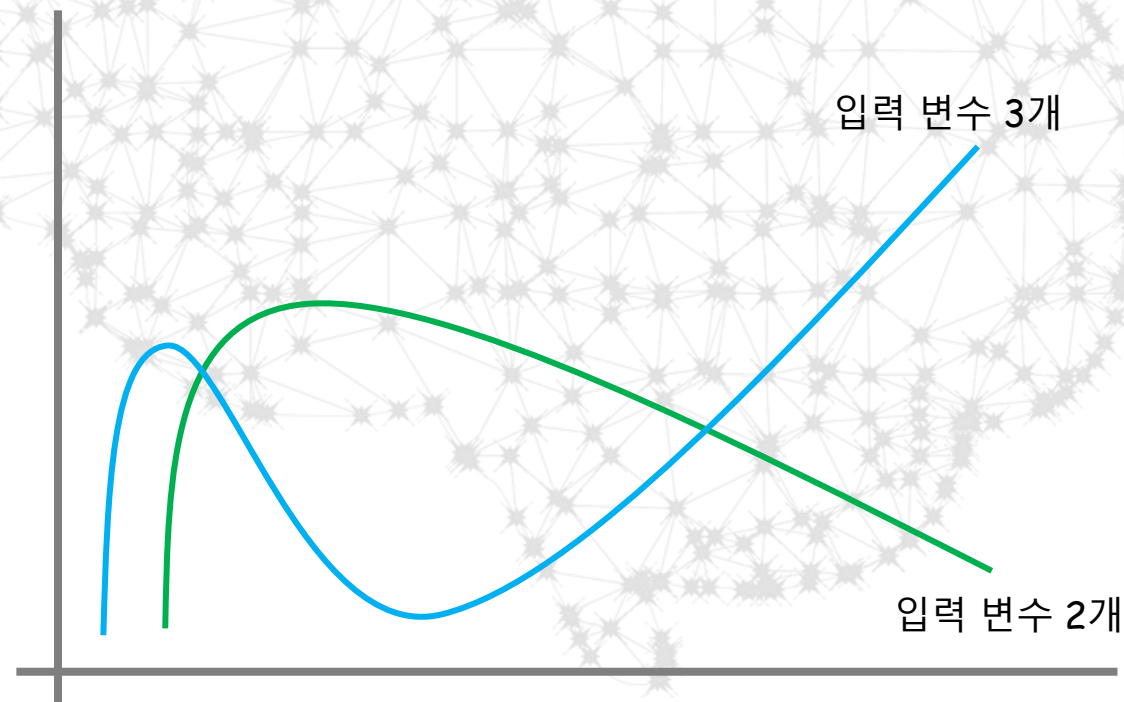
Regression

Linear Regression

- 선형(Linear) 모델은 학습 속도와 예측이 빠르다는 장점이 있다.
- 매우 큰 data set 또는 희소 data set 에서도 잘 동작 한다.
- 입력 변수가 1개인 선형 회귀의 예측은 1차원 적인 직선 긋기가 된다.

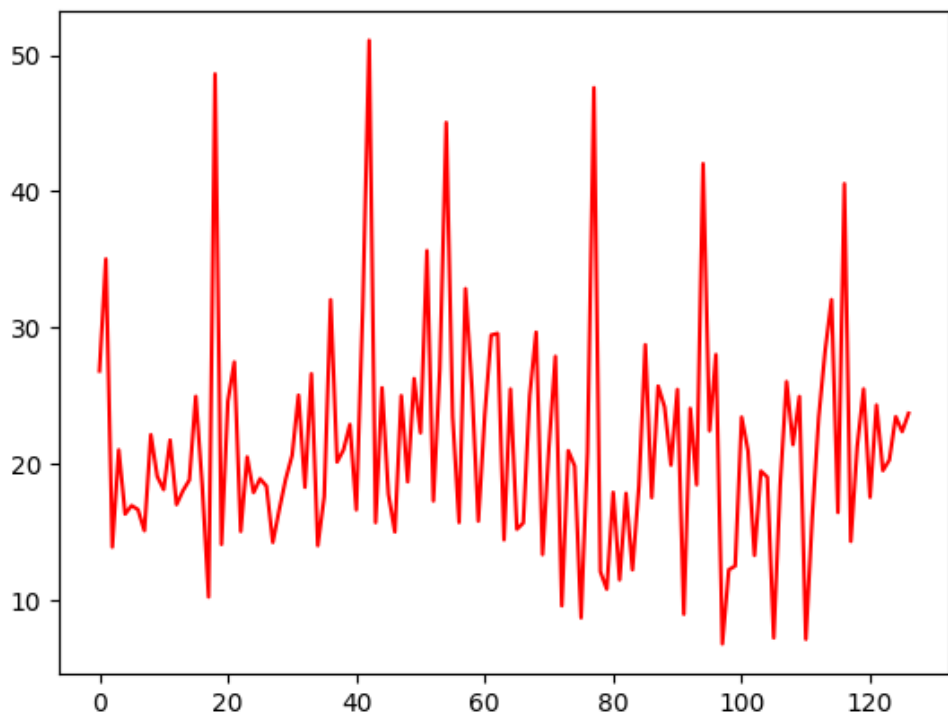


실습파일 : 09_reg.ipynb

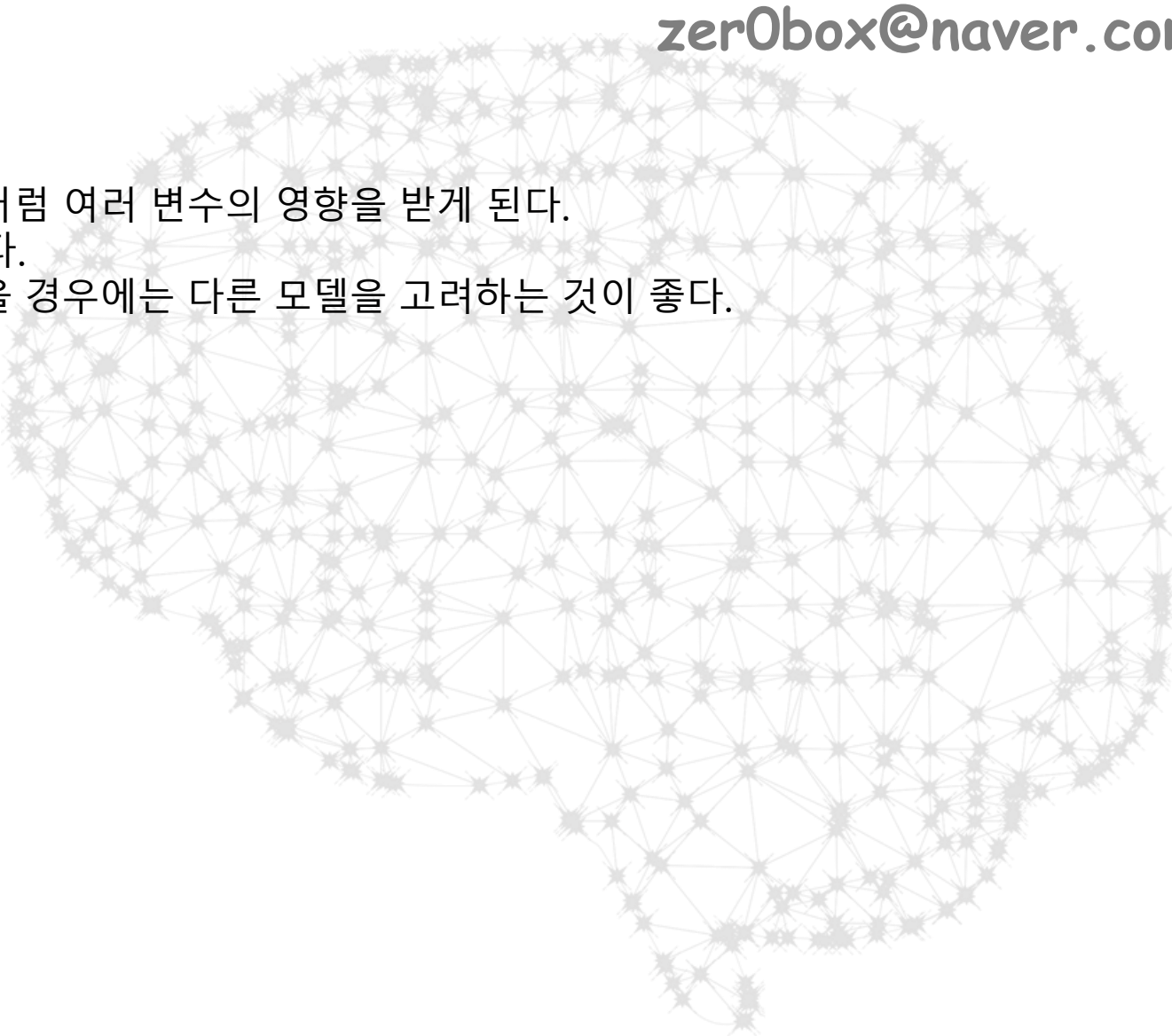


Linear Regression

- 입력 변수(속성)가 여러 개인 선형 회귀는 아래처럼 여러 변수의 영향을 받게 된다.
- 이는 과대 적합(overfitting) 이 될 가능성이 높다.
- 이처럼 너무 많은 입력 변수(속성)를 가지고 있을 경우에는 다른 모델을 고려하는 것이 좋다.



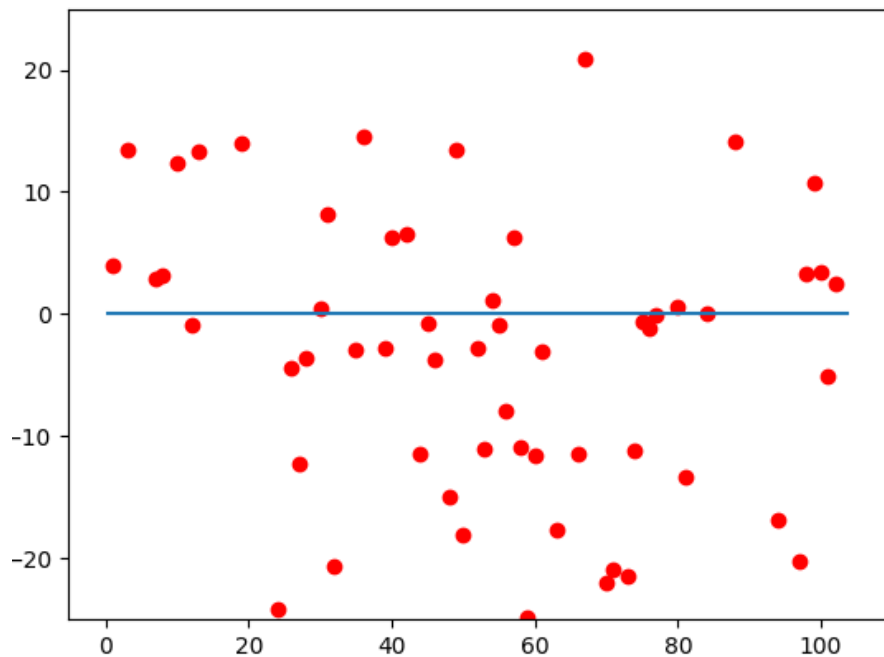
실습파일 : 09_reg.ipynb



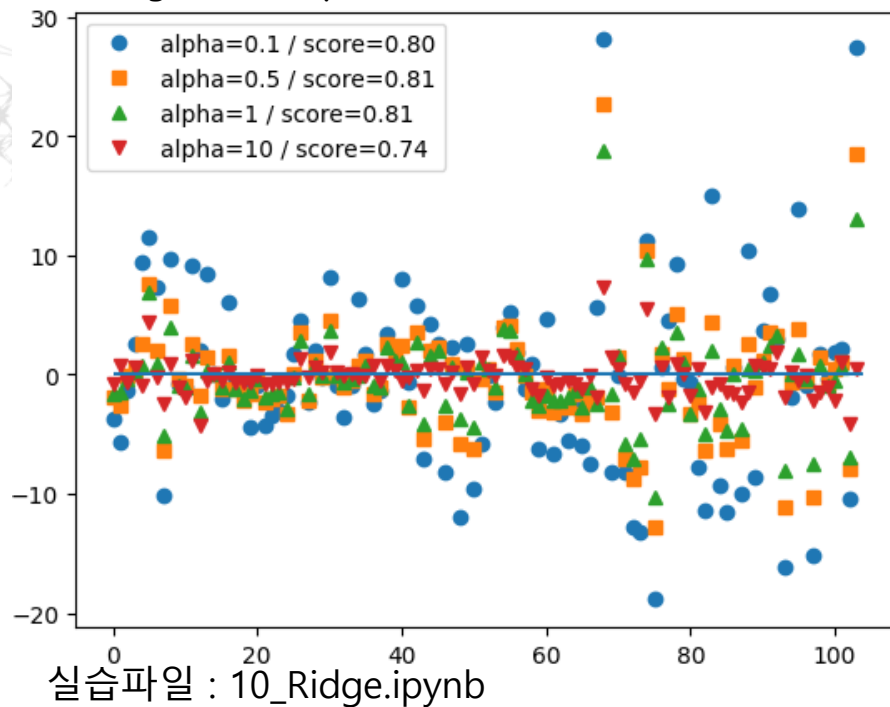
Ridge Regression

- 릿지(Ridge) 회귀 모델은 과대 적합(overfitting)을 개선하기 위해 만들어졌다.
- Alpha 라는 규제 값을 적용하여 입력 변수(속성)의 가중치를 **최소로(0에 가깝게)** 만들어 **곡선을 완만하게** 만든다.
- Ridge 에서 사용하는 이러한 규제방식을 **L2 규제**라 부른다.

- 선형 회귀 모델 속성 가중치 범위



- Ridge 모델 Alpha 값에 따른 속성 가중치 범위 변화



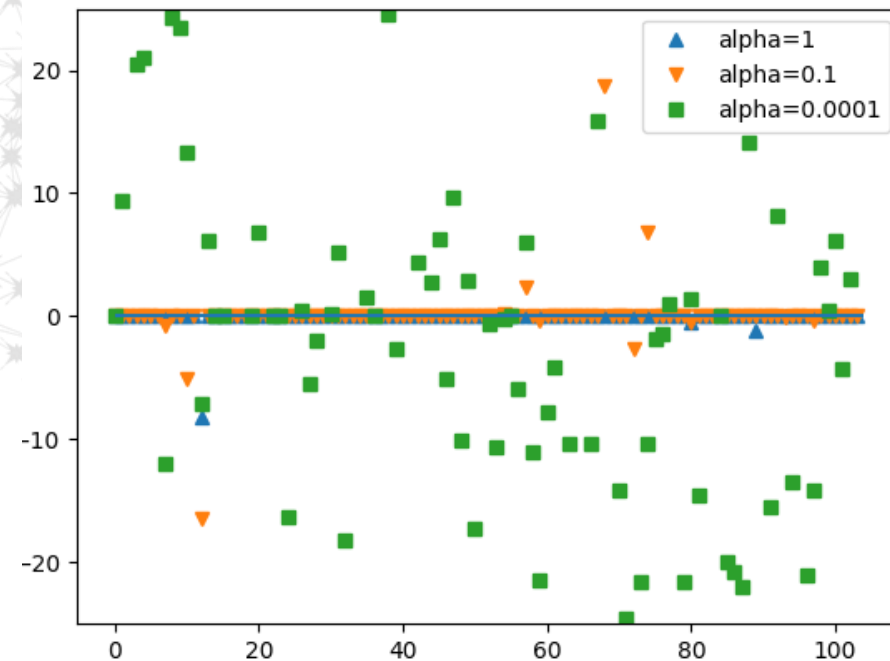
실습파일 : 10_Ridge.ipynb

Lasso Regression

- **Ridge** 회귀 모델은 Alpha 라는 규제 값을 적용하여 입력 변수(속성)의 **가중치를 0에 가깝게** 만든다.
- **Lasso** 는 Ridge 와 비슷하지만 몇몇 **특성에 대해서는 0으로** 만든다.(속성 제외)
- Lasso 에서 사용하는 이러한 규제방식을 **L1 규제**라 부른다.

사용 모델	적합한 상황
Ridge	많은 속성들 전체적으로 중요도가 비슷한 경우
Lasso	많은 속성 중 일부는 중요도가 없는 경우

- $\alpha=1$ 일 경우 계수 대부분이 0이거나 작아진다.
- $\alpha=0.1$ 이 되면 $\alpha=1$ 대비 0보다 커지는 빈도가 늘어난다.
- $\alpha=0.0001$ 이 되면 거의 규제 받지 않는다.



실습파일 : 11_Lasso.ipynb

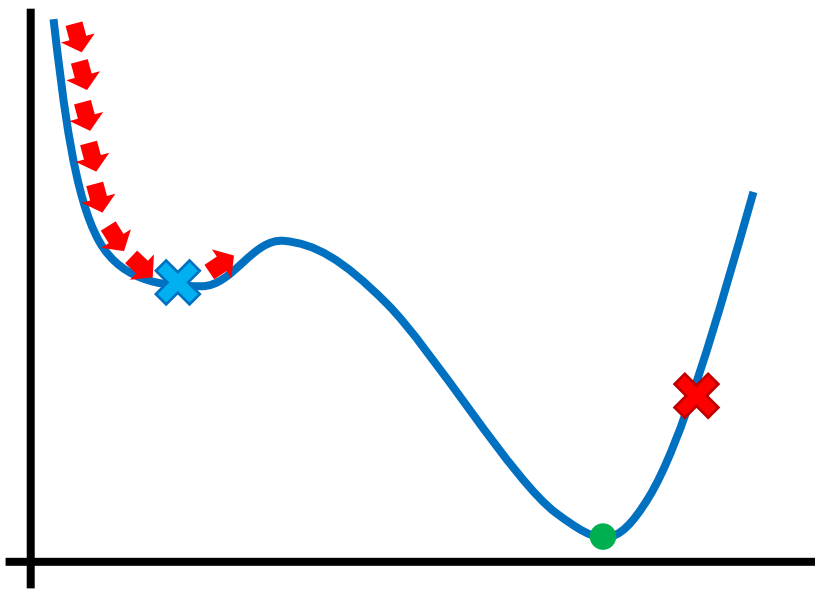
Lasso Regression

- Lasso 모델 사용시 `max_iter` 를 100,000 이상으로 정하게 되어있다.
- 이는 경사 하강법에 사용할 최대 반복 수 이다.

```
model = Lasso(alpha=0.01, max_iter=100000).fit(x_train,y_train)
```

실습파일 : 11_Lasso.ipynb

• 경사 하강 알고리즘



- 경사 하강은 오차를 줄여 나가는 알고리즘
- 산에서 내려오듯이 오차가 낮아지는 각도로 움직임
- 탐색 횟수가 **부족하면 최적점에 도달하지 못할 수 있음**
- 탐색 횟수가 너무 **많으면 최적점을 지나칠 수 있음**

Elastic Net

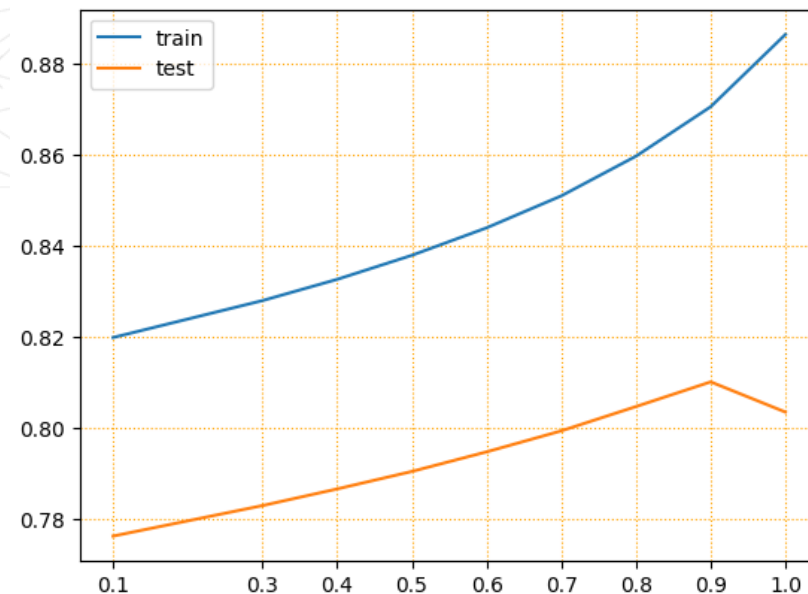
- Elastic Net 은 Lasso(L1) 와 Ridge(L2) 규제를 결합한 모델이다.
- `l1_ratio` 를 통해 L1(값 1) 과 L2(값 0) 의 규제 비율을 조절 할 수 있다.
- `Alpha` 를 통해 가중치 조절도 가능 하다

```
model = ElasticNet(alpha=0.01, l1_ratio=0.1, max_iter=100000).fit(x_train, y_train)
```

실습파일 : 11_Lasso.ipynb

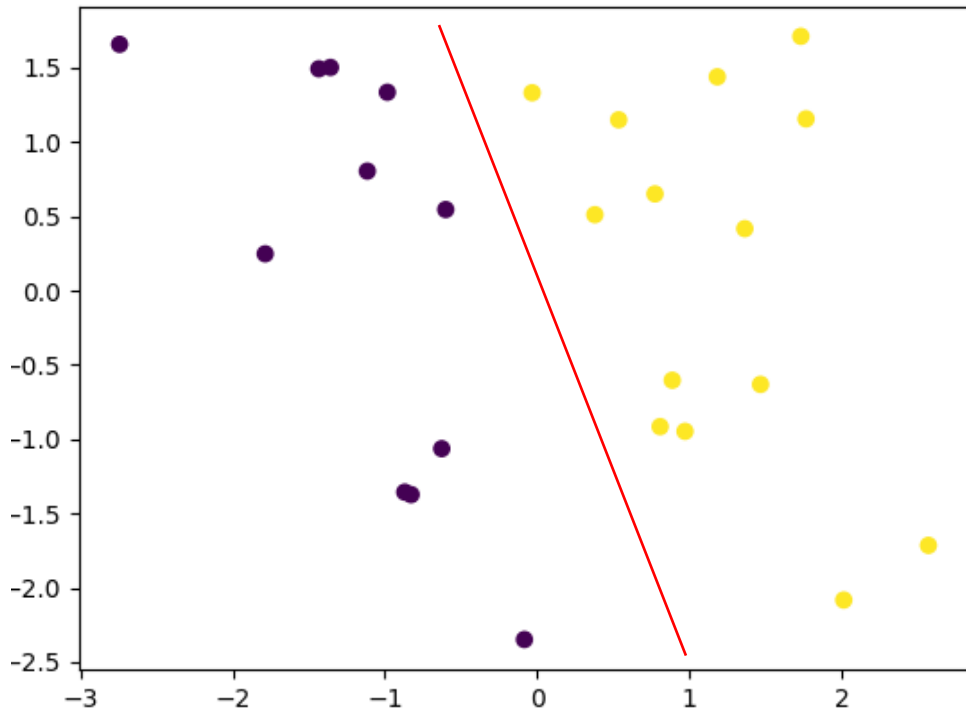
가중치 조절

L1 규제 비율



Logistic Regression

- 선형 분류 알고리즘은 특정한 결정 선을 그어 분류를 하는 것 이다.
- 그 구분의 수에 따라 Logistic Regression 과 Support Vector Machine 으로 구분 된다.
- Logistic Regression 은 binary classification(이진 분류) 에 적합하다.



```
LogisticRegression(C=100).fit(x_train, y_train)
```

실습파일 : 12_Logistic.ipynb

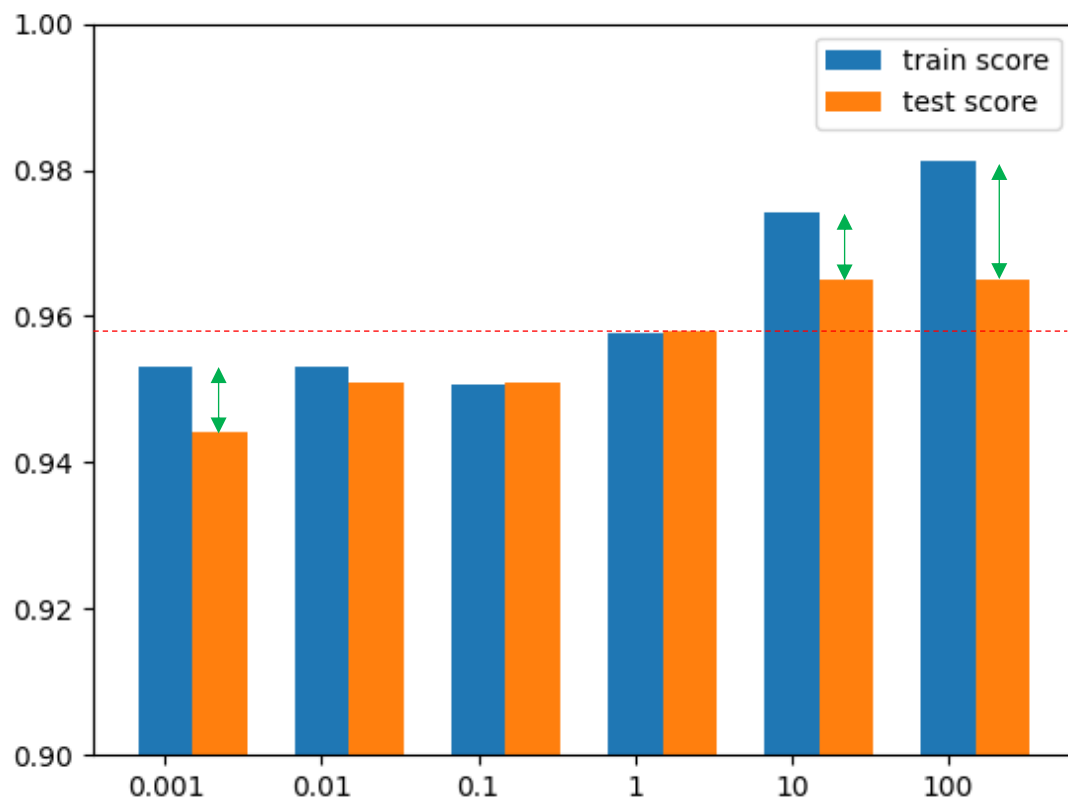
C : 규제강도(0.01~100)
강할수록(0.01) 과소적합 우려
약할수록(100) 과대적합 우려

Logistic Regression

- penalty 속성은 규제 종류를 지정 할 수 있다. l1, l2, elasticnet, none

```
LogisticRegression(C=0.1, penalty='l2', max_iter=100000).fit(x_train, y_train)
```

실습파일 : 12_Logistic.ipynb



- c=1 은 score 가 높고 과 적합 우려도 작아서 안정적
- c=100 은 c=10 에 비해 train 차이가 크므로 과 적합 우려

7. Grid Search CV

- 각 알고리즘에는 여러가지 인자 값을 가지고 있다.
- 우리는 이런 값을 얼마가 적절한지 알기 위해서는 끊임없이 값을 넣어봐야 한다.
- 하지만 `GridSearchCV` 를 활용하면 보다 쉽게 확인 할 수 있다.
- **사용알고리즘**, **최적을 찾고싶은 파라미터값**, **교차검증세트**

```
alphas = [0.001,0.01,0.1,1,10]
iters = [1000,5000,10000,50000,100000]
```

```
params = {'max_iter':iters,'alpha':alphas}
grid = ms.GridSearchCV(Ridge(), param_grid=params, cv=5).fit(x,y)
print(f'Ridge Best Param : {grid.best_params_}')
print(f'Ridge Best Score : {grid.best_score_}')
```

실습파일 : 13_grid_search_cv.ipynb

요약(Summary)

Regression 의 장점

- 학습 속도와 예측이 빠르다.
- 매우 큰 데이터 셋이나 희소 데이터 셋 모두에게 잘 동작 한다.

Regression 의 단점

- 직선으로만 표현 되다 보니 특정 문제에 취약하다(SVM 에서 다룰 예정)
- 데이터의 속성(변수)이 적을 경우 단순해 진다.

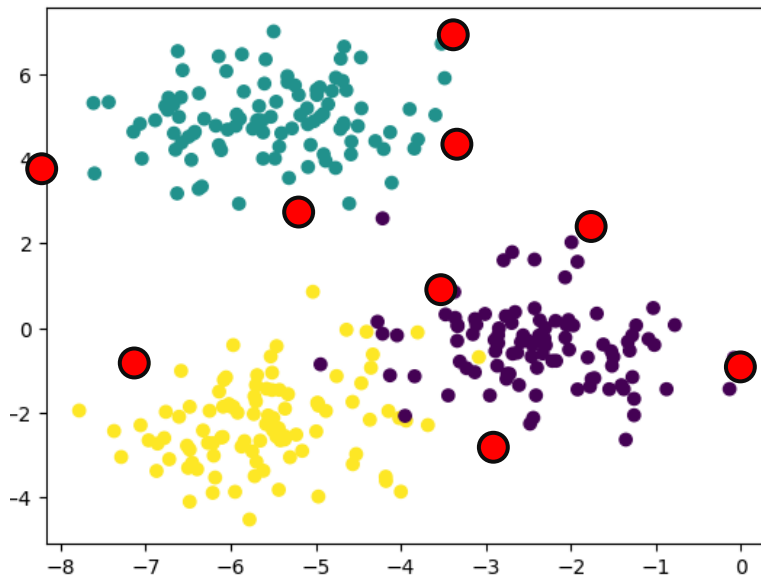


Support Vector Machine

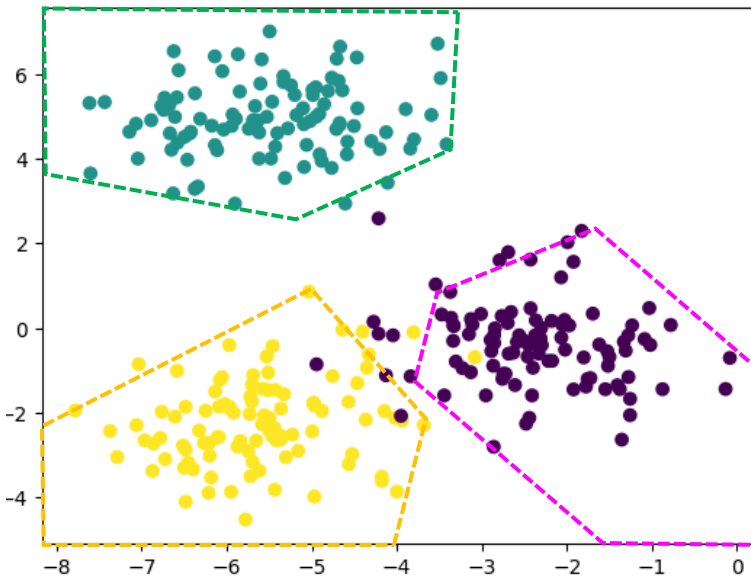
SVM(Support Vector Machine) - 분류 용 선형 모델

- SVM 은 다중 분류가 가능 하다. 분류, 회귀 및 이상 값 탐지에 사용 된다.
- (1)특정 집단의 가장자리 포인트(Support Vector) 들을 중심으로
- (2) 사이 공간(margin) 을 정의
- (3) 하고 그 공간의 가운데를 HyperPlane 으로 설정하여 분류

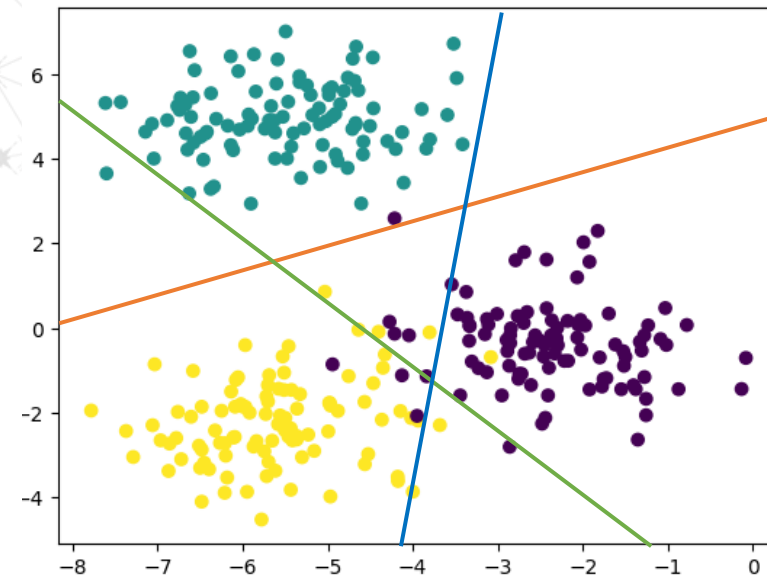
(1)



(2)



(3)



SVM(Support Vector Machine) - 분류 용 선형 모델

- SVM 은 dual 이라는 매개변수를 통해 최적화 알고리즘을 선택 할 수 있다.

```
svc = LinearSVC(C=0.1,penalty='l2', dual=True).fit(x_train,y_train)
```

실습파일 : 14_svm.ipynb

옵션	설명
C	규제강도(값이 낮을수록 규제강도가 높아진다.)
Penalty	규제종류 - L1,L2(기본),Elastic
dual	False(기본) - $n_samples(샘플) > n_features(특성)$ 일 경우 적합(데이터의 수가 입력 항목 보다 많을 경우)
	True - $n_samples(샘플) < n_features(특성)$ 일 경우 적합(데이터의 수보다 입력 항목이 많을 경우)

이름

이름	나이	성별	급여

coefficient 와 intercept 의 활용

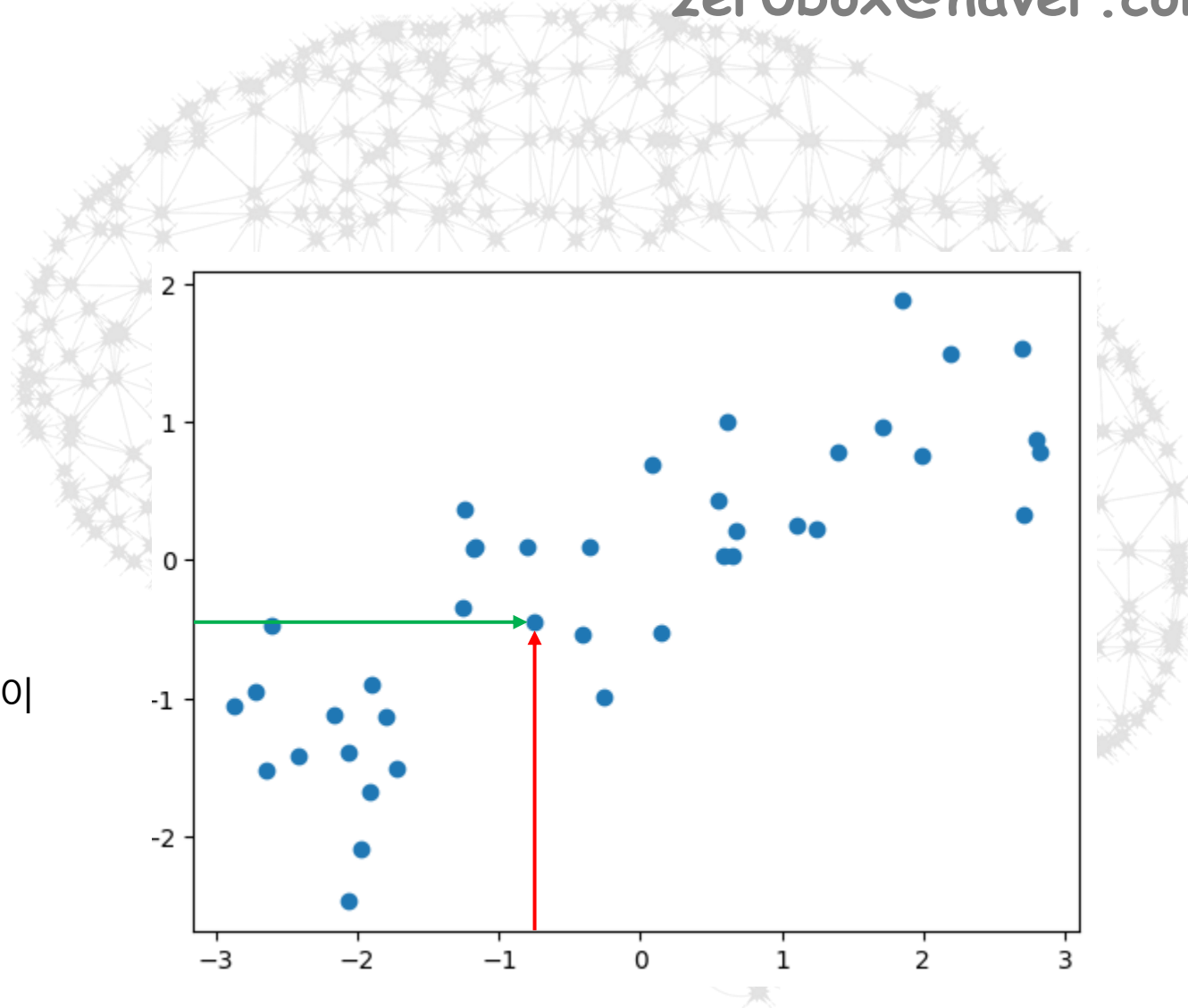
- 머신 러닝의 기본은 아래의 식과 같다.

$$y = ax + b$$

→ -0.75275929 를 넣었더니

→ -0.44822072637593674 이 나왔다.

- 이때 학습을 통해 **a** 와 **b** 의 값을 찾아내는 것이 머신 러닝의 기초이다.



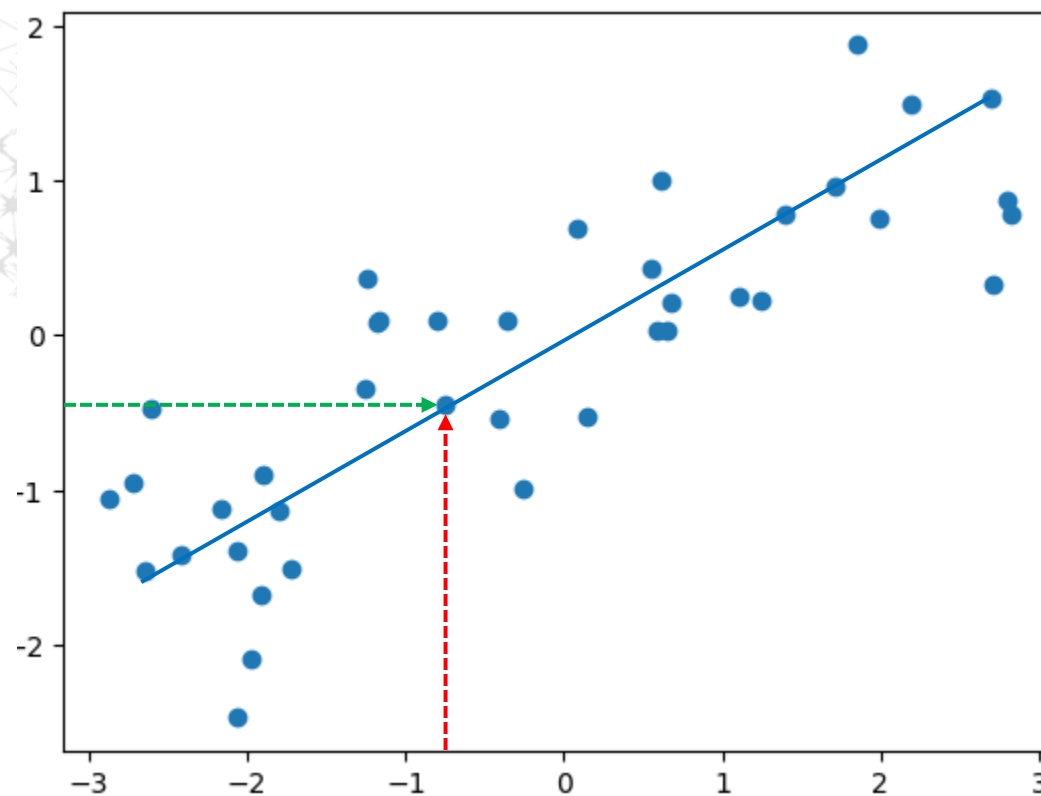
coefficient 와 intercept 의 활용

- 여기서 a 는 coefficient, 계수 이며 b 는 intercept, 절편 이다.
- 그래프 에서 coefficient 는 기울기를 나타내며, intercept 는 방향을 나타낸다.

$$y = ax + b$$

coefficient

intercept



실습파일 : 15_coef.ipynb

coefficient 와 intercept 의 활용

- 아래와 같은 분류에서 어떻게 경계선을 긋는지 확인해 보자

```
svc = LinearSVC(C=0.1,penalty='l2', dual=True).fit(x_train,y_train)
```

- 계수가 이전과 다르게 2개가 나왔다.
- 보통 구분을 위한 선을 그을 때는 2개의 계수를 가지게 된다.

계수 (coef) : [-0.27943197 0.07382683]
절편 (intercept): 0.8792356010798675

- 그리고 계수가 1개 이상이면 공식이 달라진다.

- 특성 값이 1개(x)인 경우

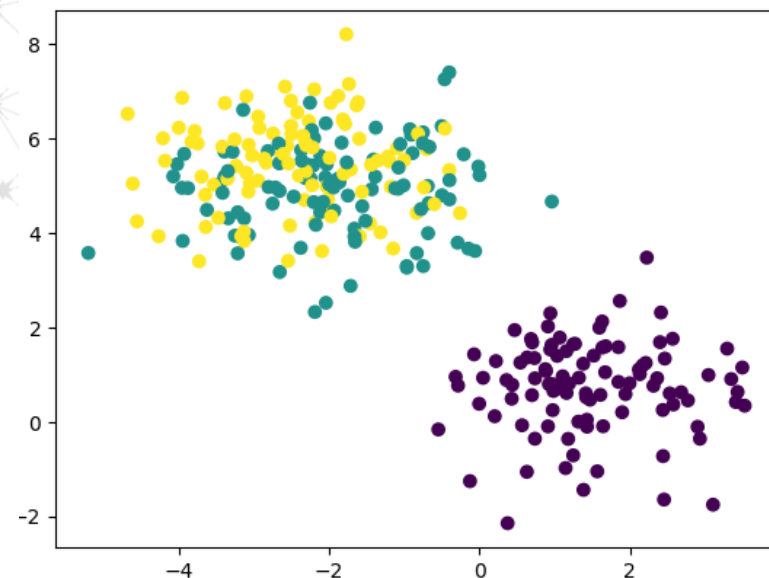
답 = 계수 * 특성 + 절편

$$y = \text{coef_} * x + \text{intercept_}$$

- 특성값이 n개(x,y)인 경우

답 = (계수[0] * 특성1 + 계수[1] * 특성2 + ...) + 절편

$$y = ((\text{coef_}[0] * x[0]) + (\text{coef_}[1] * x[1])) + \text{intercept}$$



coefficient 와 intercept 의 활용

- 우리가 학습한 데이터를 보면 $x[0]$, 과 $x[1]$ 로 각각의 좌표를, 답으로 나온 y 값으로 색상을 표현했다.

```
plt.scatter(x[:,0],x[:,1],c=y)
```

- 입력 값(특성)이 2개이며, coef 가 2개 이므로 다른 공식을 사용한다.

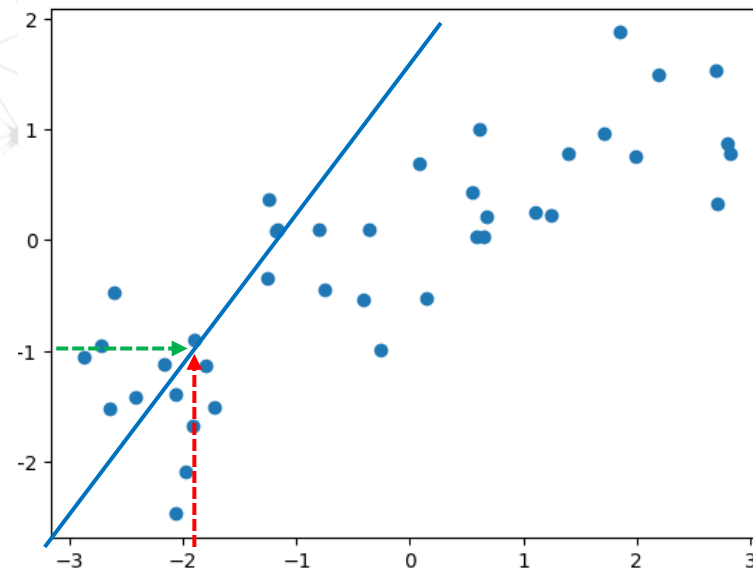
$$(coef[0] * x[0] + coef[1] * x[1]) + intercept = y$$

- 그런데 현재 우리가 알고 싶은 것은 분류하는 기울기 이므로 $x[0]$ 이 증가할 때마다 $x[1]$ 이 얼마나 증가하는 것이다.
- 그래서 이제 우리는 $x[0]$ 을 통해 $x[1]$ 의 값을 알고 싶어 진다.

$$(coef[0] * \boxed{x[0]} + coef[1] * \boxed{x[1]}) + intercept = 0$$

X

Y



coefficient 와 intercept 의 활용

- 여기서 부터는 산수의 영역

(1) $(\text{coef}[0] * \mathbf{x} + \text{coef}[1] * \mathbf{y}) + \text{intercept} = 0$

$$(\text{coef}[1] * \mathbf{y}) = -(\text{intercept} + \text{coef}[0] * \mathbf{x})$$

(2) $(\text{coef}[1] * \mathbf{y}) = -(\text{intercept} + \text{coef}[0] * \mathbf{x})$

$$\mathbf{y} = -(\text{intercept} + \text{coef}[0] * \mathbf{x}) / \text{coef}[1]$$

coefficient 와 intercept 의 활용

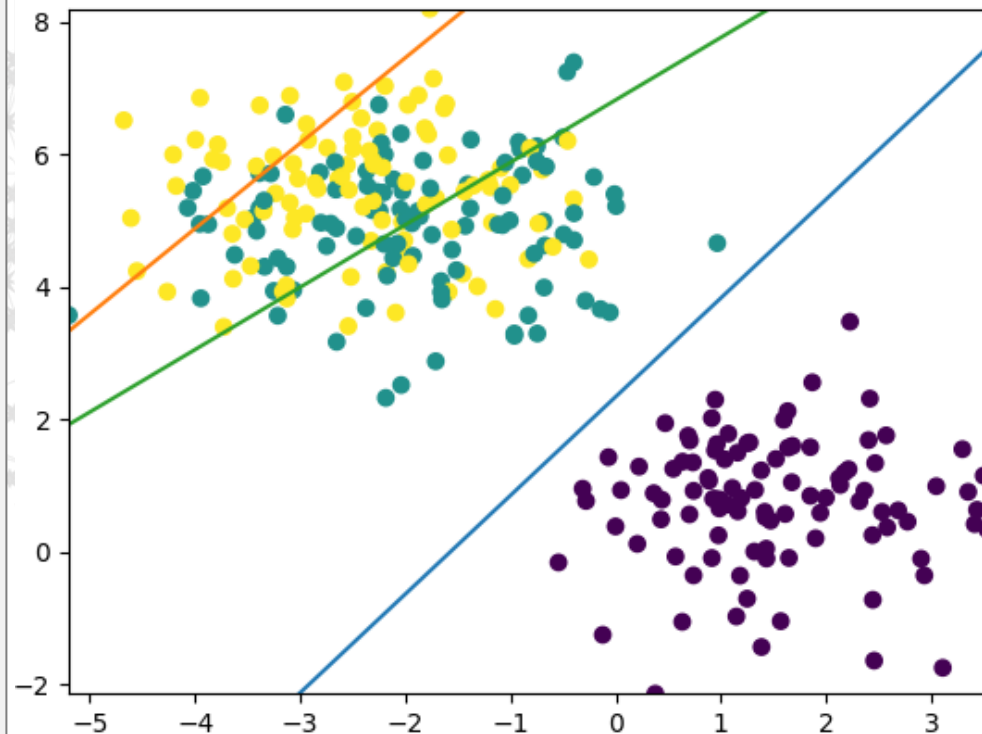
- 결과적으로 아래처럼 얻어낸 계수와 절편 값으로 경계선을 그릴 수 있다.

```
print(f'계수(coef) : {svc.coef_[0]}') # 기울기
print(f'절편(intercept):{svc.intercept_[0]}') # 방향
plt.scatter(x[:,0],x[:,1],c=y)
line = np.linspace(-10,10)

for coef,inter in zip(svc.coef_,svc.intercept_):
    print(coef,inter)
    plt.plot(line, -(inter + coef[0]*line) / coef[1])

plt.xlim(x[:,0].min(),x[:,0].max())
plt.ylim(x[:,1].min(),x[:,1].max())
plt.show()
```

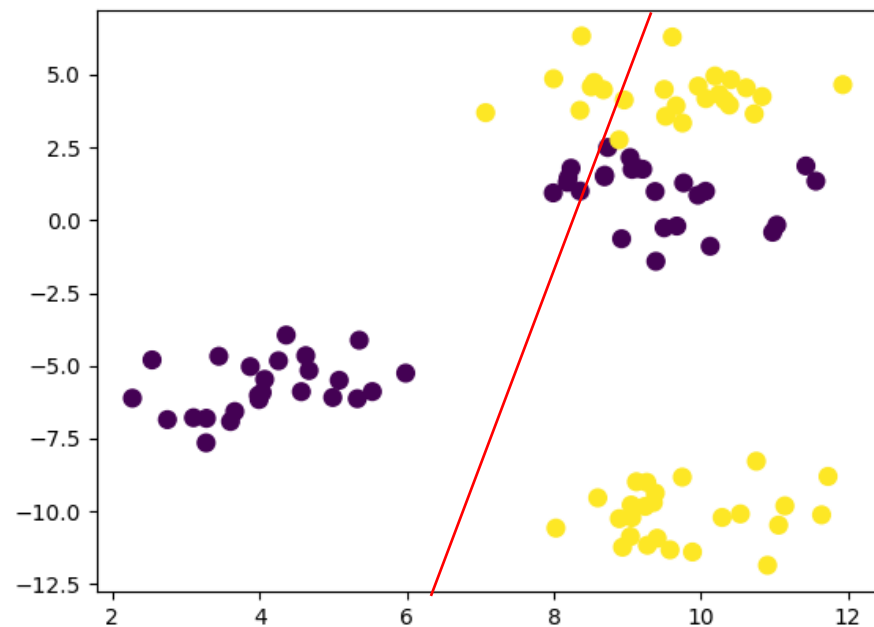
실습파일 : 15_coef.ipynb



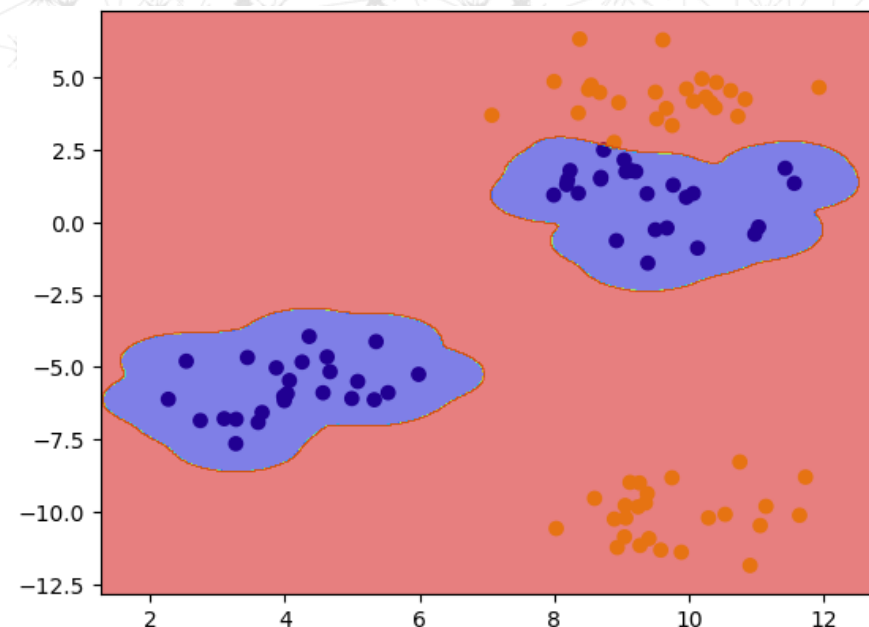
Kernel support vector machine

- 선형 벡터 머신은 아래 처럼 XOR 문제에 대한 대처가 되지 않는다.
- 이럴 때 커널 벡터머신을 통해 분류해 줄 수 있다.

• 선형 벡터



• 커널 벡터



3. Kernel support vector machine

- kernel 기법을 통해 비선형의 특성을 추가해 줄 수 있다.

```
model = svm.SVC(kernel='sigmoid', C=10, gamma=20).fit(x,y)
```

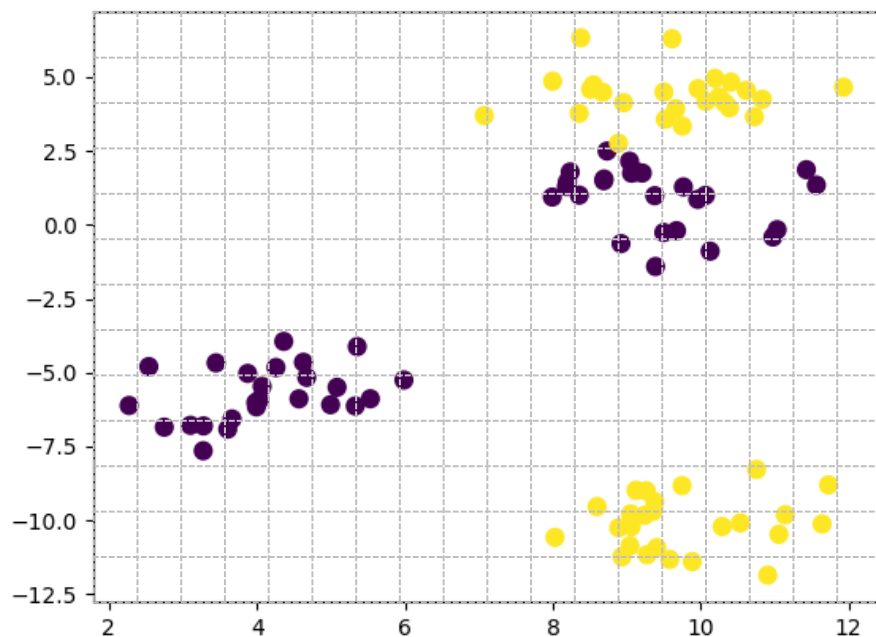
kernel	설명
Linear(선형)	데이터가 이미 어느 정도 선형적으로 분리될 때 사용(계산 속도가 가장 빠름)
poly(다항커널)	데이터를 다항식 형태로 확장(이미지 처리 등에서 가끔 사용)
rbf(가우시안커널)	가장 많이 쓰이는 기본 커널(복잡한 데이터도 잘 분류)
sigmoid(시그모이드커널)	신경망의 활성화 함수와 비슷

옵션	설명
C(Cost)	오차허용 정도(클수록 엄격 해져 경계가 복잡해짐)
gamma	영향력이 짧아져서 결정 경계가 구불구불해지고 예민해짐
degree	다항커널에서 차원 추가(1:직선, 2:포물선,...)

Kernel support vector machine

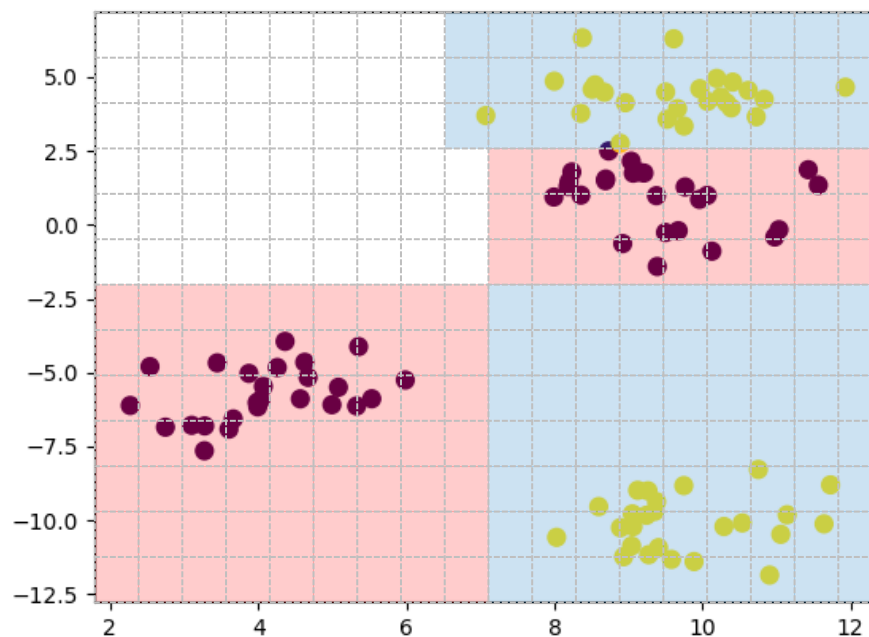
- 커널 머신은 선을 긋는 형태가 아니므로 영역 형태로 표현하기 위한 시각화가 필요하다.
- 그래서 다음과 같은 형태로 그림을 그릴 예정이다.

1. 표를 커다란 격자형태로 나눈다.



2. 격자 단위로 x 와 y 좌표를 넣어 예측치 z 를 얻는다.

3. z 를 이용해 영역 색상을 칠한다.



Kernel support vector machine

1. 최소값(예:0)과 최대값(예:3)을 이용해 격자모양 구조의 배열을 생성한다.

```
pos_x = np.arange(x_min, x_max, 0.02)
pos_y = np.arange(y_min, y_max, 0.02)
xx, yy = np.meshgrid(pos_x, pos_y)
```

실습파일 : 16_kernel.ipynb

0	1	2	3	3	3	3	3
0	1	2	3	2	2	2	2
0	1	2	3	1	1	1	1
0	1	2	3	0	0	0	0

xx

yy

2. `ravel()` 함수를 통해 1차원 배열로 만들어 `np.c_` 를 통해 합쳐 준다.

```
val = np.c_[xx.ravel(), yy.ravel()]
```

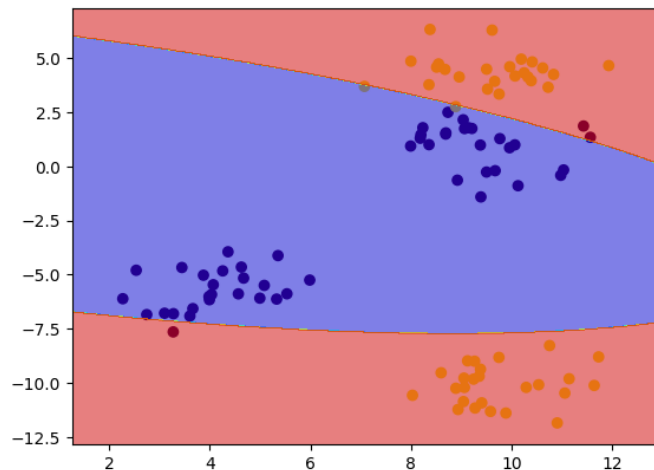


3. 그렇게 받아낸 `val` 을 이용해 예측을 시키고, 2차원 배열 형태로 만들어 `contour` 를 그리도록 한다.

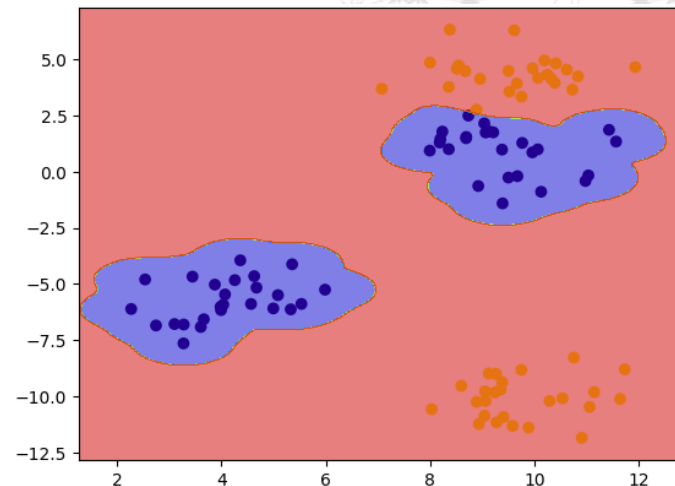
```
z = model.predict(val)  #[0,0,0,...,1,1,1]
z = z.reshape(xx.shape)#[[0,0,0],[1,1,1],...]
plt.scatter(x[:,0],x[:,1],c=y)
plt.contourf(xx,yy,z,cmap='jet', alpha=0.5)
plt.show()
```

Kernel support vector machine

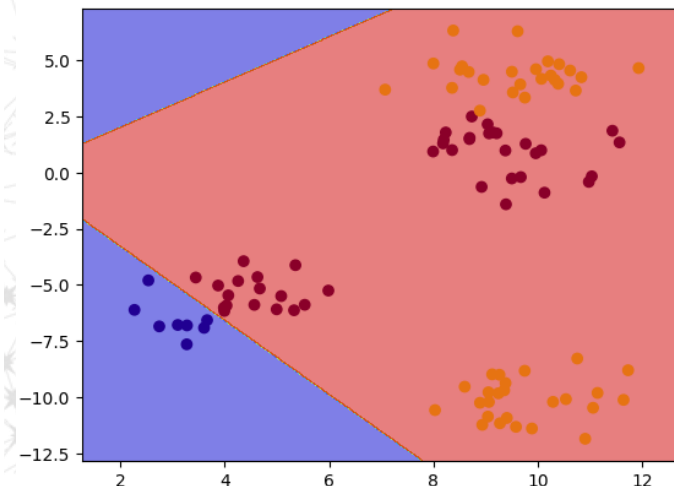
- 각 kernel 과 파라미터 값에 따라 변경되는 경계를 확인해 보자



- kernel='poly'
- C=10
- degree=2
- gamma=20



- kernel='rbf'
- C=10
- gamma=5



- kernel='sigmoid'
- C=10
- gamma=20

Kernel support vector machine

- kernel 기법은 유용하지만 아래 특징을 가지고 있다.

no	설명
1	Sample 이 많은 경우 잘 맞지 않는다.(10,000 개 기준)
2	데이터 전 처리에 신경 써야 한다.(특성의 단위가 비슷해야 한다.)
3	매개변수 설정에 신경 써야 한다.(매개변수에 따라 크게 변경됨)

요약(Summary)

SVM 의 장점

- 다양한 데이터 셋에서도 잘 동작 한다.
- 데이터 속성이 많고 적음에 큰 성능 차이를 보이지 않는다.

SVM 의 단점

- 데이터의 전 처리와 매개변수에 신경 써야 한다.
- Sample 이 너무 많을 경우 정확도가 떨어진다.



Naive Bayes

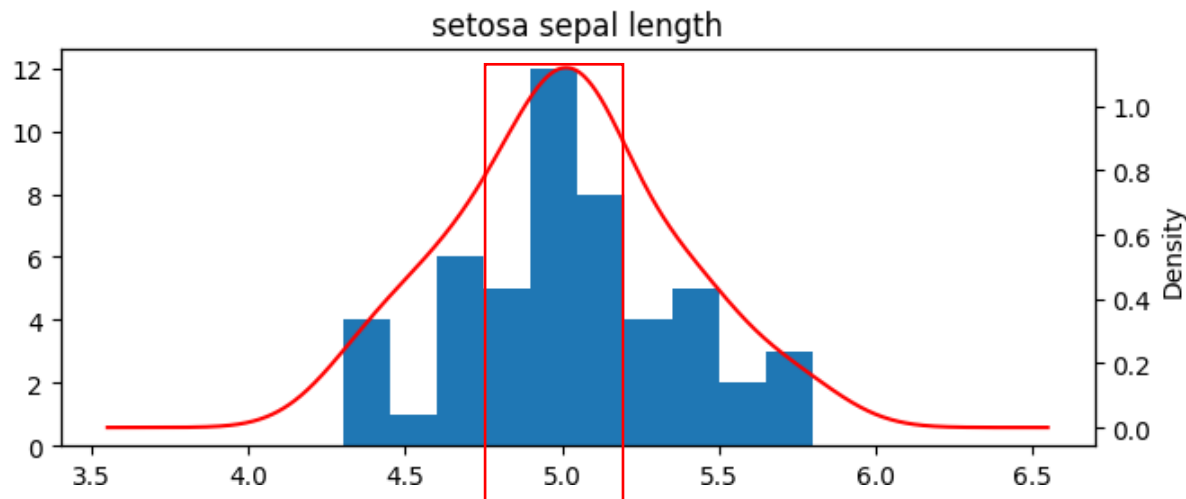
Naive Bayes Classifier

- 나이브 베이즈 분류기는 베이즈 정리(Bayes theorem)를 이용한 분류 모델 이다.
 - 큰 데이터에 대해서 훈련 속도는 빠르지만 일반화 성능은 느린 편이다.
 - 나이브 베이즈 분류기는 각 속성 간에 독립성이 보장되어야 한다.(속성 간 상관 관계가 없어야 한다)
예) 단어 들 중에서 한 단어가 변경된다고 다른 단어와의 연관성이 생기거나 사라지지 않는다.
- Scikit learn 에서 제공한 분류 기

classifier	descript
GaussianNB	연속적인 데이터에 주로 적용
BernoulliNB	Binary 속성 데이터에 주로 적용(텍스트를 이용한 스팸 여부 분류에 사용)
MultinomialNB	카운트 데이터에 주로 적용(텍스트 빈도를 통한 분류에 사용)

Gaussian Naive Bayes

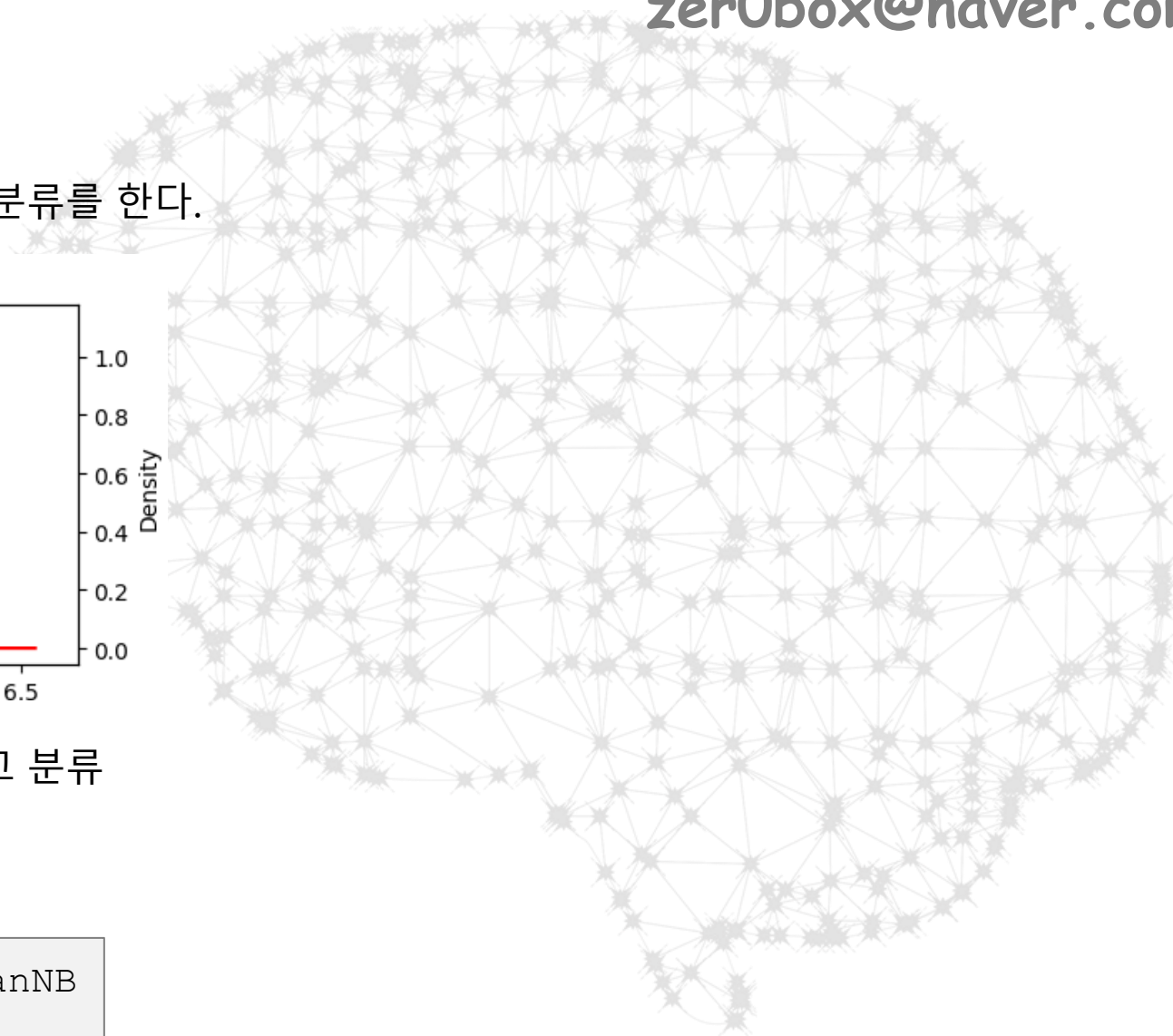
- GaussianNB 는 가우시안 정규 분포를 활용 하여 분류를 한다.



- 꽃받침 길이가 5cm 정도 되면 **setosa** 종이라고 분류

```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB().fit(x_train, y_train)
```

실습파일 : 17_Gaussian.ipynb

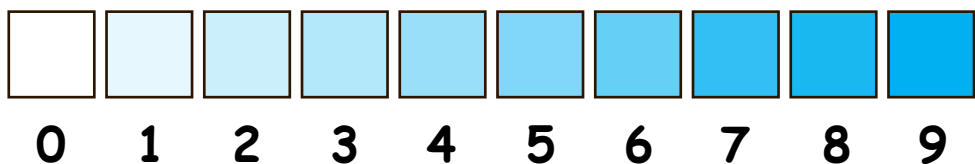


Bernoulli Naive Bayes

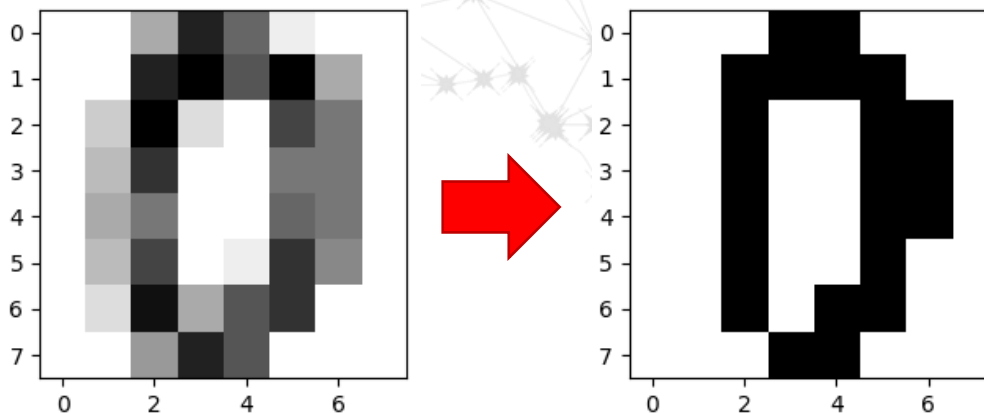
- BernoulliNB 는 베르누이 확률 분포를 활용 하여 분류를 한다.

classifier	descript
베르누이 실행	성공 과 실패 두가지 결과만 존재하는 시행, 각 시행은 독립적이다.(예: 동전의 앞 뒷면)
베르누이 확률 변수	베르누이 실행 결과를 1과 0으로 바꾼 것(예: 앞면-1/뒷면-0)
베르누이 확률 분포	확률 변수에 대한 결과 분포(딱 한번만 시행 했을 경우를 고려)
베르누이 이항 분포	확률 변수에 대한 결과 분포(연속적으로 n번 시행 했을 경우를 고려)

* 투명도 몇 부터가 파란색이라고 할 수 있는가?



* 손 글씨 구분(정확히 어떤 글자인가?)



Multinomial Naive Bayes

- MultinomialNB 는 counter data(예: 문장에 나타난 단어의 횟수) 에 적용하면 적합 하다.
- 분류 별 특성의 평균을 계산한다.
- 어떤 이메일의 text 가 입력되었을 때 스팸일 확률을 알아 보려고 한다.
- 이때 두 확률을 구해 높은 쪽으로 분류를 하게 된다.

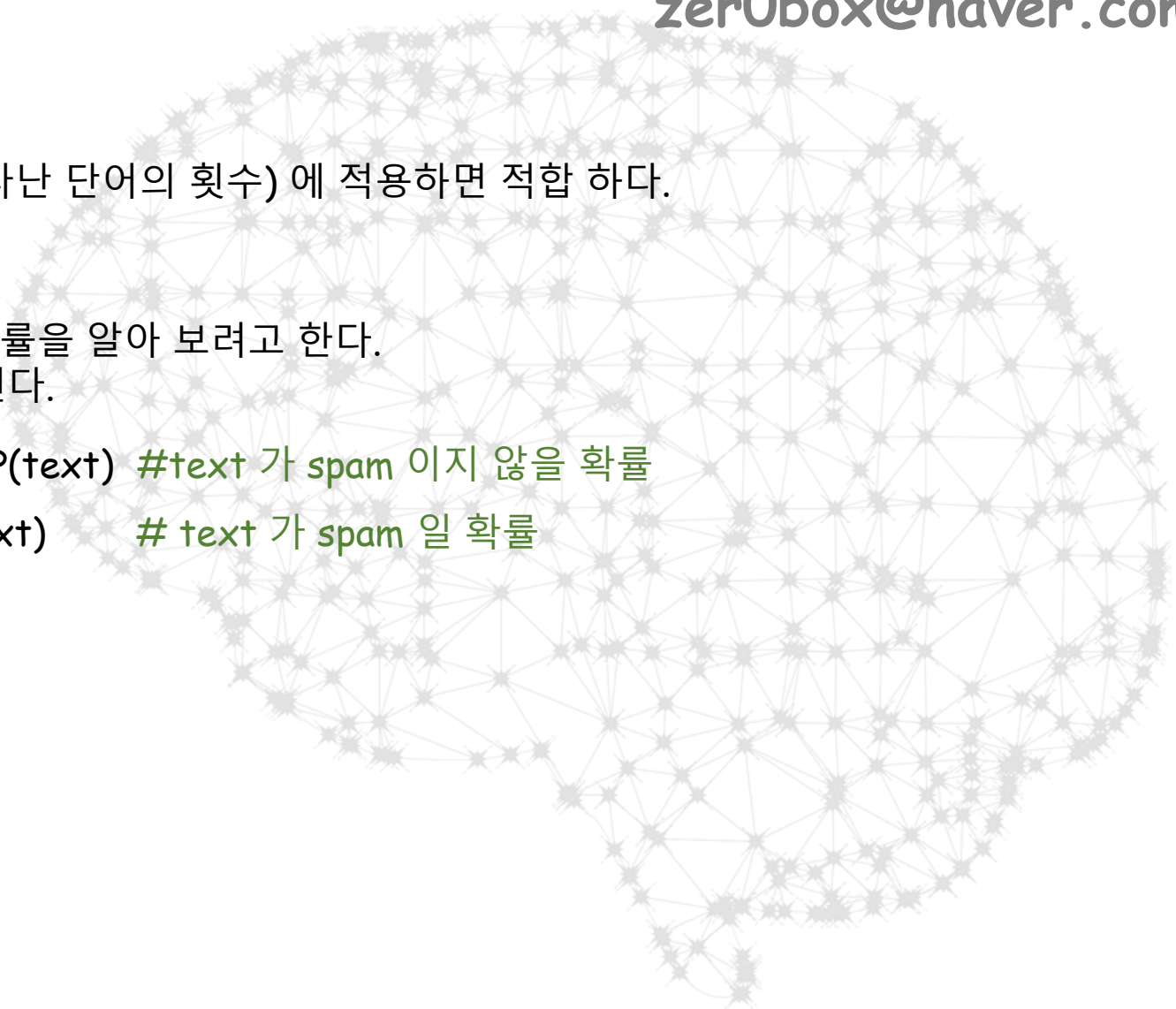
$$P(\sim\text{spam}|\text{text}) = P(\text{text}|\sim\text{spam}) * P(\sim\text{spam}) / P(\text{text}) \quad \# \text{text} \text{ 가 spam 이지 않을 확률}$$

$$P(\text{spam}|\text{text}) = P(\text{text}|\text{spam}) * P(\text{spam}) / P(\text{text}) \quad \# \text{text} \text{ 가 spam 일 확률}$$

- 부모가 같으니 $p(\text{text})$ 는 제거해 줘도 된다.

$$P(\sim\text{spam}|\text{text}) = P(\text{text}|\sim\text{spam}) * P(\sim\text{spam})$$

$$P(\text{spam}|\text{text}) = P(\text{text}|\text{spam}) * P(\text{spam})$$



Multinomial Naive Bayes

- 이공식을 가지고 다음을 구분해 보자
- **You free lottery** 는 SPAM 인가?

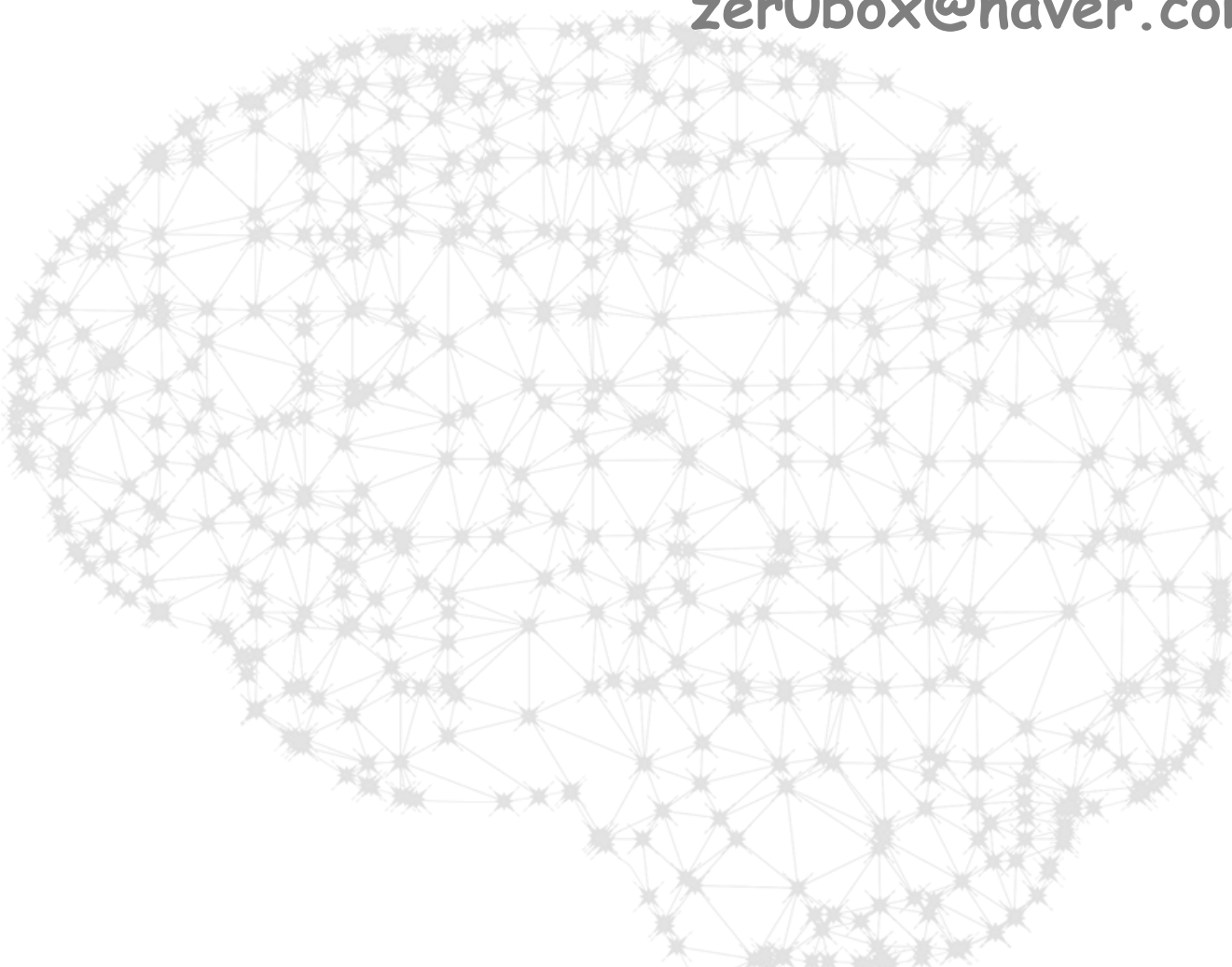
학습 데이터

text	classification
me free lottery	Spam
free get free you	Spam
you free scholarship	Normal
free to contact me	Normal
you won award	Normal
you ticket lottery	Spam

- 3 단어에 대해서 다음과 같이 계산하게 됨

$P(\text{text 가 normal}) = P(\text{word1 가 normal 확률}) * P(\text{word2 가 normal 확률}) * P(\text{word3 가 normal 확률}) * P(\text{normal 비율})$

$P(\text{text 가 spam}) = P(\text{word1 가 spam 확률}) * P(\text{word2 가 spam 확률}) * P(\text{word3 가 spam 확률}) * P(\text{spam 비율})$



Multinomial Naive Bayes

You free lottery 는 SPAM 인가?

You : spam=2, normal=2

Free : spam=2, normal=2

Lottery : spam=2

Total word : 10(me, free, lottery, get, scholarship, to, contact, won award, you, ticket)

Total mail : 6

text	classification
me free lottery	Spam
free get free you	Spam
you free scholarship	Normal
free to contact me	Normal
you won award	Normal
you ticket lottery	Spam

Multinomial Naive Bayes

normal 일 확률

you 는 총 10번 출현 중 2번이 normal 이었다. (2/10)

free 는 총 10번 출현 중 2번이 normal 이었다. (2/10)

lottery 는 총 10번 출현 중 0번이 normal 이었다. (0/10)

위 단어들이 출현한 메일 중 spam 판정을 3건 받았다. (3/6)

$$P(\text{text가 normal}) = (2/10) * (2/10) * (0/10) * (3/6) = 0$$

spam 일 확률

you 는 총 10번 출현 중 2번이 spam 이었다. (2/10)

free 는 총 10번 출현 중 2번이 spam 이었다. (2/10)

lottery 는 총 10번 출현 중 2번이 spam 이었다. (2/10)

위 단어들이 출현한 메일 중 spam 판정을 3건 받았다. (3/6)

$$P(\text{text가 spam}) = (2/10) * (2/10) * (2/10) * (3/6) = 0.02$$



normal 일 확률 0, spam 일 확률 0.02 이므로...
you free lottery 는 spam 이다.

Vector

- 컴퓨터는 문자를 이해할 수 없어 문자 분석이 어렵다.
- 이때 문자를 컴퓨터가 이해하고 분석할 수 있는 숫자리스트로 바꾸는 작업을 **Vector** 화 라고 한다.
- 아래 3가지 문장에 대해 각각의 **Vector** 화가 어떻게 이루어지는지 확인해 보자

문장 1: I love apple / 문장 2 : I love banana / 문장 3 : I hate apple

1. **CountVectorizer** : 문서별로 등장하는 단어들을 모아 몇 번 나타나는지 세는 방식

apple	banana	hate	I	love
-------	--------	------	---	------



문장1	1	0	0	1	1
문장2	0	1	0	1	1
문장3	1	0	1	1	0

Vector

문장 1: I love apple / 문장 2 : I love banana / 문장 3 : I hate apple

2. DictVectorizer : 딕셔너리 형태 데이터를 수치형 벡터로 바꾸는 방식(딕셔너리 형태로 되어있어야만 한다.)

{'I': 1, 'love': 1, 'apple': 1},

{'I': 1, 'love': 1, 'banana': 1},

{'I': 1, 'hate': 1, 'apple': 1}



	I	apple	banana	hate	love
문장1	1	1	0	0	1
문장2	1	0	1	0	1
문장3	1	1	0	1	0

- One-Hot Encoding: 해당 속성이 존재하면 1, 없으면 0

Vector

문장 1: I love apple / 문장 2 : I love banana / 문장 3 : I hate apple

3. TfidfVectorizer : CountVectorizer 에 희소성 가중치를 곱한다.

apple	banana	hate	I	love
-------	--------	------	---	------



문장1	1	0	0	1	1
문장2	0	1	0	1	1
문장3	1	0	1	1	0

I : 3번 등장(가중치 0.3)
Apple, love : 2번 등장(가중치 0.5)
banana, hate : 1번 등장(가중치 1)



문장1	1×0.5	0×1	0×1	1×0.3	1×0.5
문장2	0×0.5	1×1	0×1	1×0.3	1×0.5
문장3	1×0.5	0×1	1×1	1×0.3	0×0.5

Vector

- 학습 데이터는 `fit_transform()` 을, 시험 데이터는 `transform()` 을 사용 한다.

벡터함수 호출

```
Counter = text.CountVectorizer()
```

데이터를 훈련(`fit`)을 위한 벡터화(`fit_transform`)

```
vx = counter.fit_transform(x_train)
```

```
multi = MultinomialNB().fit(vx, y_train)
```

결과 확인시 벡터화(`transform`)

```
vx_test = counter.transform(x_test)
```

```
print(f'test : {multi.score(vx_test, y_test)}')
```

실습파일 : 20_vectorizer.ipynb

function	description
<code>fit_transform()</code>	연습 데이터 값의 평균이(mean) 0 분산이(variance) 1 이 되도록 조정(수치의 차를 크지 않도록 스케일링)
<code>transform()</code>	연습 데이터에 적용한 스케일을 테스트 데이터에도 적용

6. Pipe line

- 앞에서 했던 내용을 **pipe line** 을 활용하면 일부 코드를 생략 할 수 있다.
- 즉, **pipe line** 을 사용 하면 전처리 와 학습 과정을 한번에 처리 할 수 있게 된다.
- 시각화 작업을 할 때는 쓰지 않는 것이 좋다.

```
model1 = Pipeline([  
    ('vect', text.CountVectorizer()),  
    ('model', MultinomialNB())  
])
```

실습파일 : 21_pipeline.ipynb

데이터 수집

데이터 Scale

알고리즘 학습

성능 테스트

요약(Summary)

Naïve Bayes 의 장점

- 노이즈 데이터가 있어도 잘 수행 한다.
- 훈련 샘플이 적어도 좋은 효과를 낸다.

Naïve Bayes 의 단점

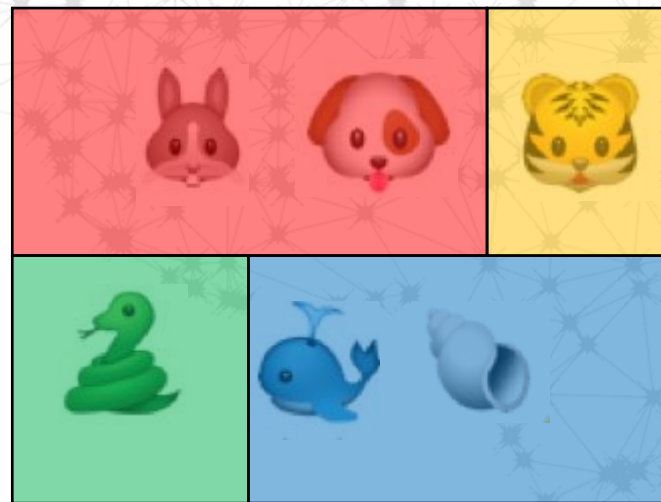
- 모든 속성이 독립적이어야 한다.(그런 데이터는 많지 않다)
- 그러다 보니 수치로 구성된 데이터에 적합하지 않다.



Decision Tree

결정 트리

- 결정 트리(Decision Tree)는 Tree 구조를 갖은 알고리즘으로 결과를 마치 스무고개 하듯이 찾아낸다.
- 또는 데이터를 특정 기준으로 점차 쪼개 가는 과정이다.
- 트리의 깊이가 깊어질수록 분할이 촘촘하게 일어나고 **overfitting** 이 발생 할 수 있다.



결정 트리

- Overfitting 을 막기 위한 전략은 아래와 같다.

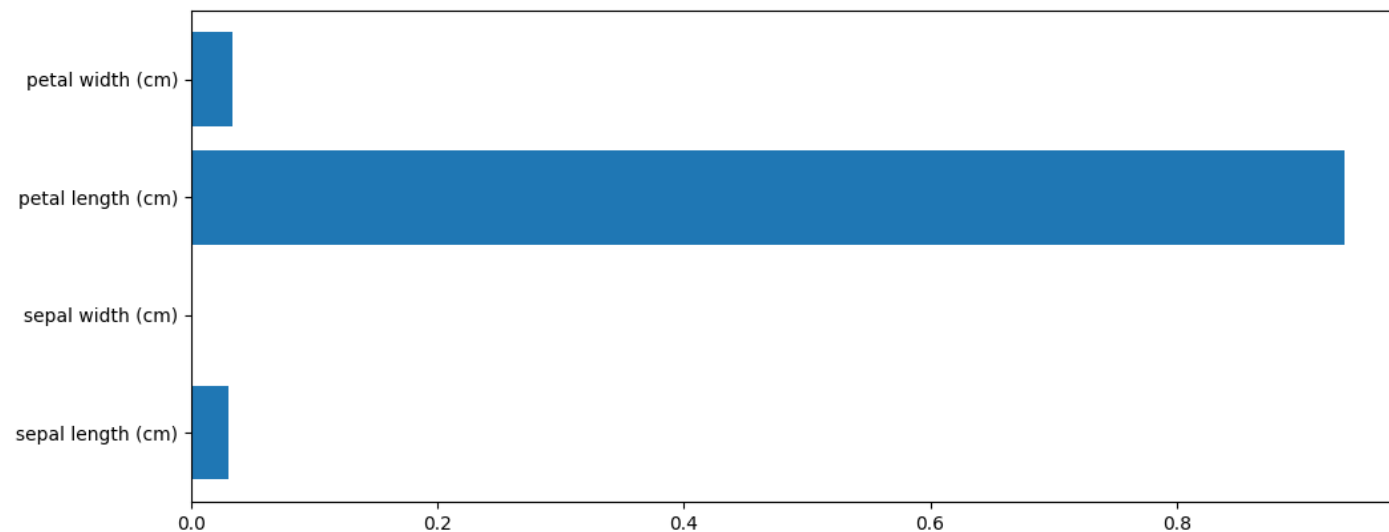
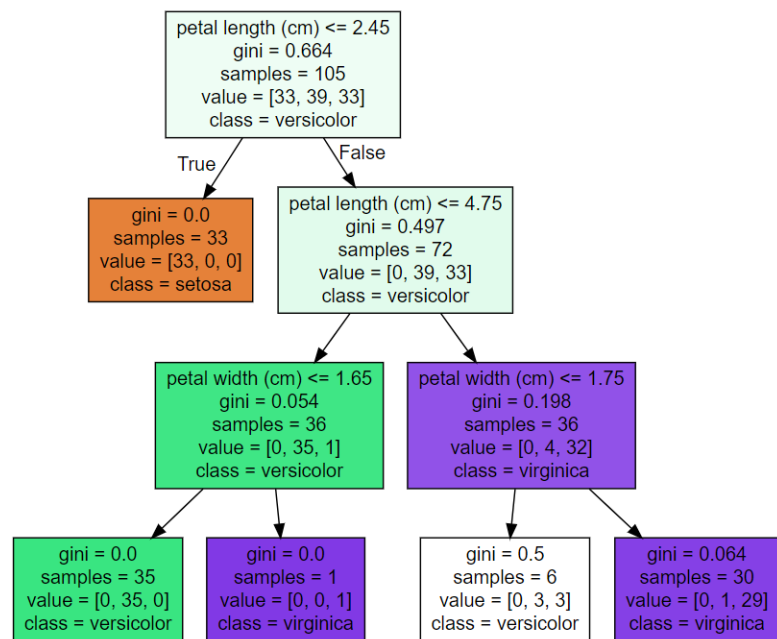
구분	설명
Pre pruning	사전 가지치기, 트리 생성을 적당히 진행하다가 중단
Post pruning	사후 가지치기, 트리 생성 후 분류 데이터가 적은 가지를 쳐내거나 병합

- Scikit learn 에서 제공하는 결정 트리는 다음과 같다.

Decision Tree Regressor	결정 회귀
Decision Tree Classifier	결정 분류

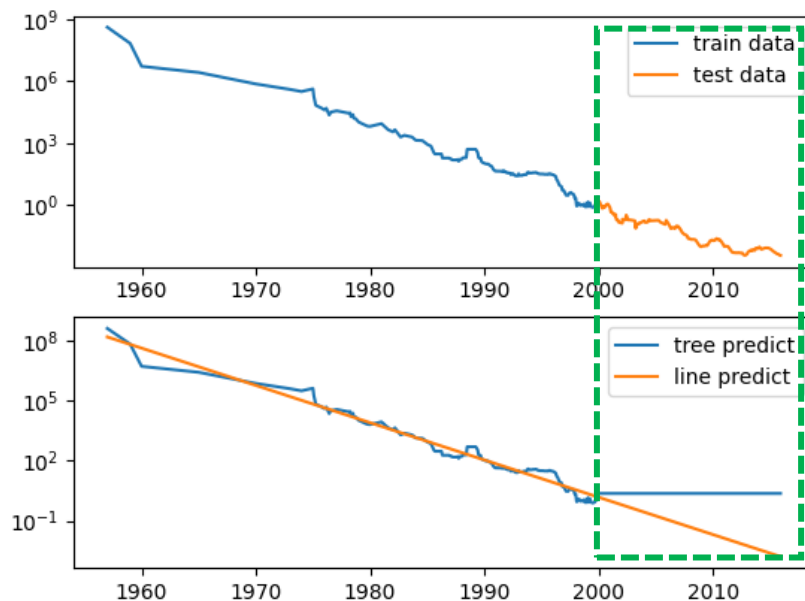
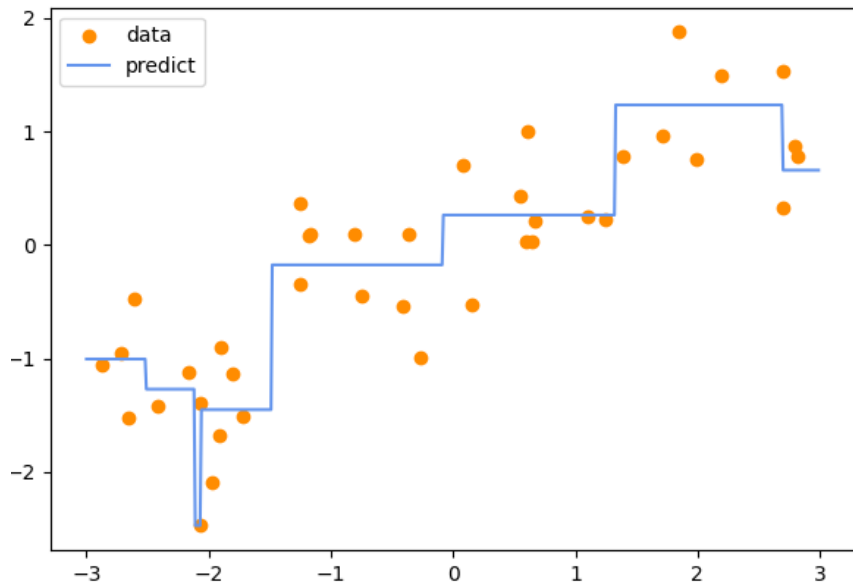
결정 분류 트리

- 특성의 값들을 분류하여 구분해 낸다.
- 이때 어떤 기준으로 구분 했는지 시각화가 가능하다.
- 어떤 속성이 결정을 내리는데 많이 참고 하였는지 확인 할 수 있다.



결정 회귀 트리

- 회귀는 연속적인 추이를 예측 할 때 유용하다.
- 하지만 훈련 데이터 범위를 크게 벗어난 데이터에 대한 예측은 할 수 없다.
- 즉, 반복되는 패턴의 데이터가 아닌 경우 예측이 쉽지 않다.
- 이는 모든 트리 기반 모델의 공통된 단점이다.



* 학습에 제공되지 않은 데이터에 대한 선형 회귀와 트리 회귀의 예측(반복되지 않고 떨어지기만 한 데이터 이므로)

요약(Summary)

Tree 의 장점

- 결정과정에 대한 쉬운 시각화
- 데이터 스케일에 구애 받지 않는다.(속성 간 값의 차가 커도 상관 없음)

Tree 의 단점

- 훈련 데이터를 벗어난 예측은 불가능 하다.
- Tree 의 depth 가 깊어지면서 overfitting 발생 확률이 높다.



Ensemble

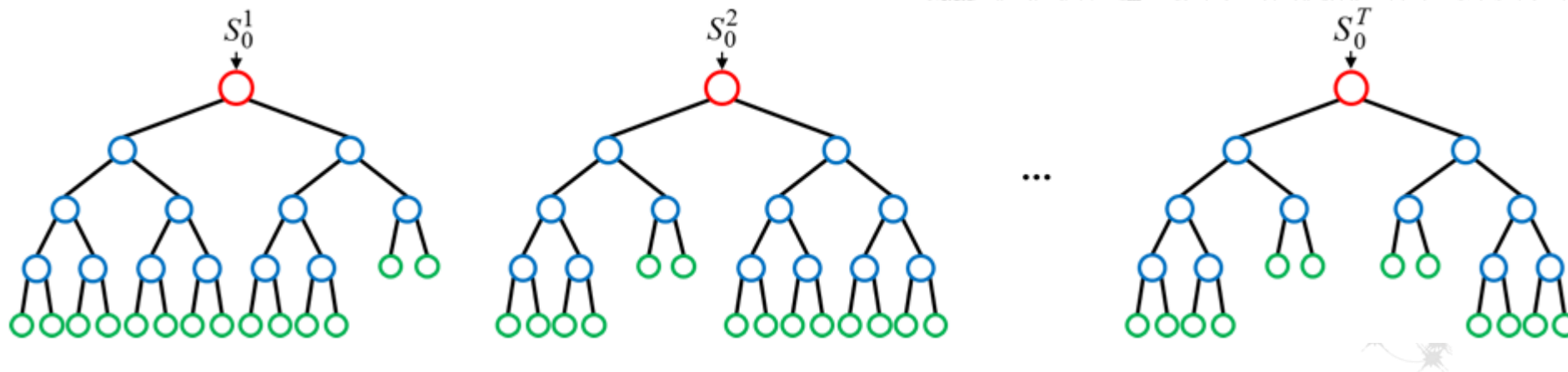
모형 결합(model combining)

- 앙상블(ensemble) 방법론 이라고 불리는 이 방법은 예측 시 복수개의 예측 모형을 결합한다.
- 모형 결합 방법은 크게 취합(aggregation) 방법론과 부스팅(boosting) 방법론이 있다.

Aggregation method	사용할 모형이 결정되어 있음	Random Forest Bagging Majority Voting(다수결)
Boosting method	사용할 모형을 점진적으로 늘림	Ada Boost Gradient Boost X.G. Boost

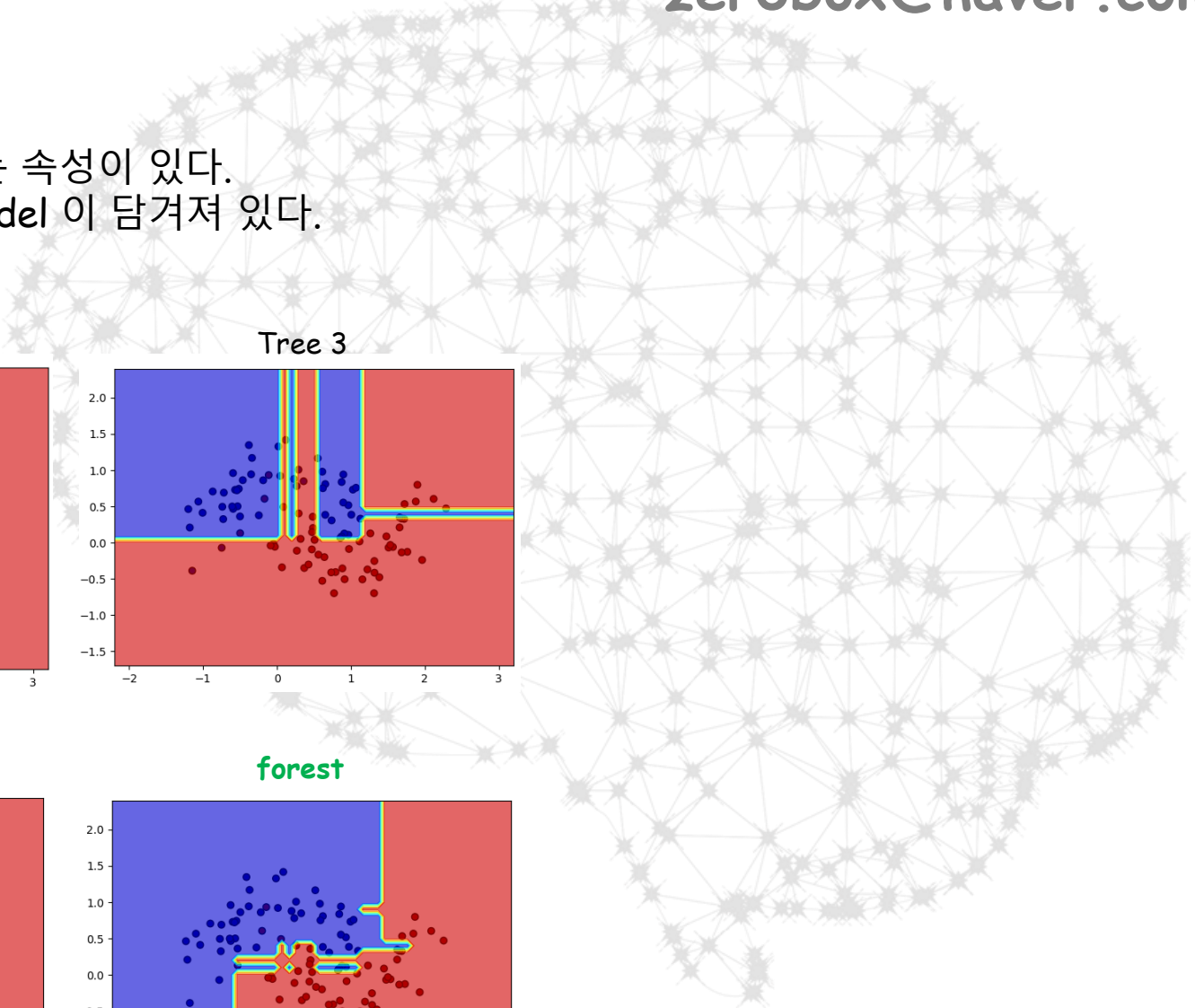
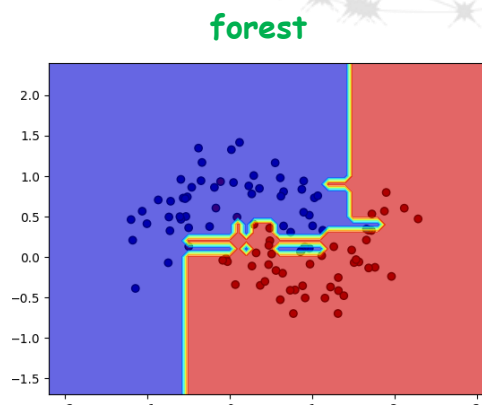
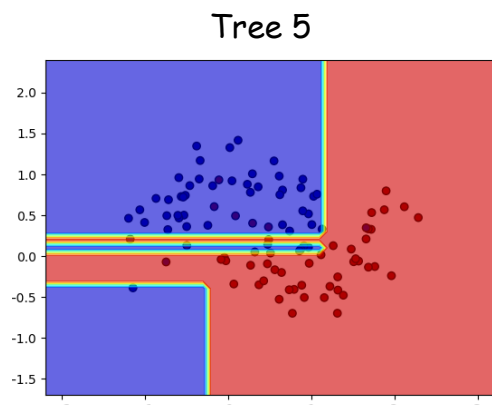
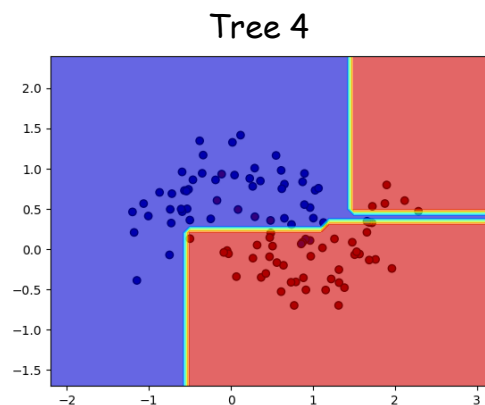
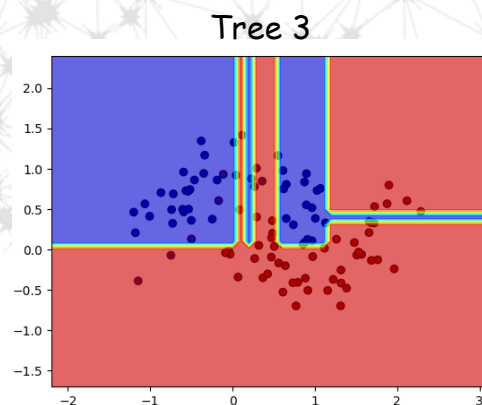
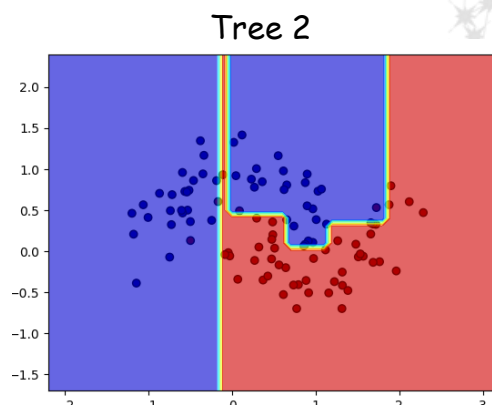
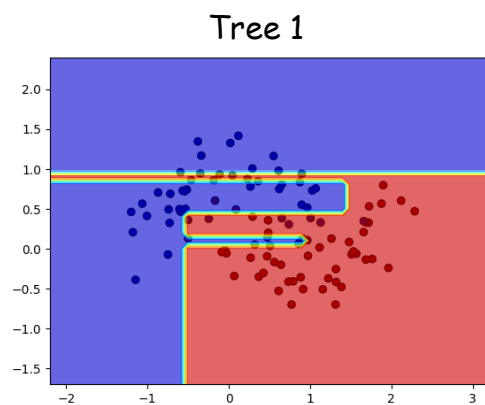
Random Forest

- 의사결정 나무(Tree) 는 우선 하나의 질문에 대해 실수로 답을 잘못하면 다시 거슬러 올라 갈 수가 없다.
- 그리고 깊이가 깊을 수록 **overfitting**의 문제가 있었다.
- 그래서 여러 나무(tree) 를 모아 숲(forest) 으로 만들어 이러한 문제를 줄이고자 했다



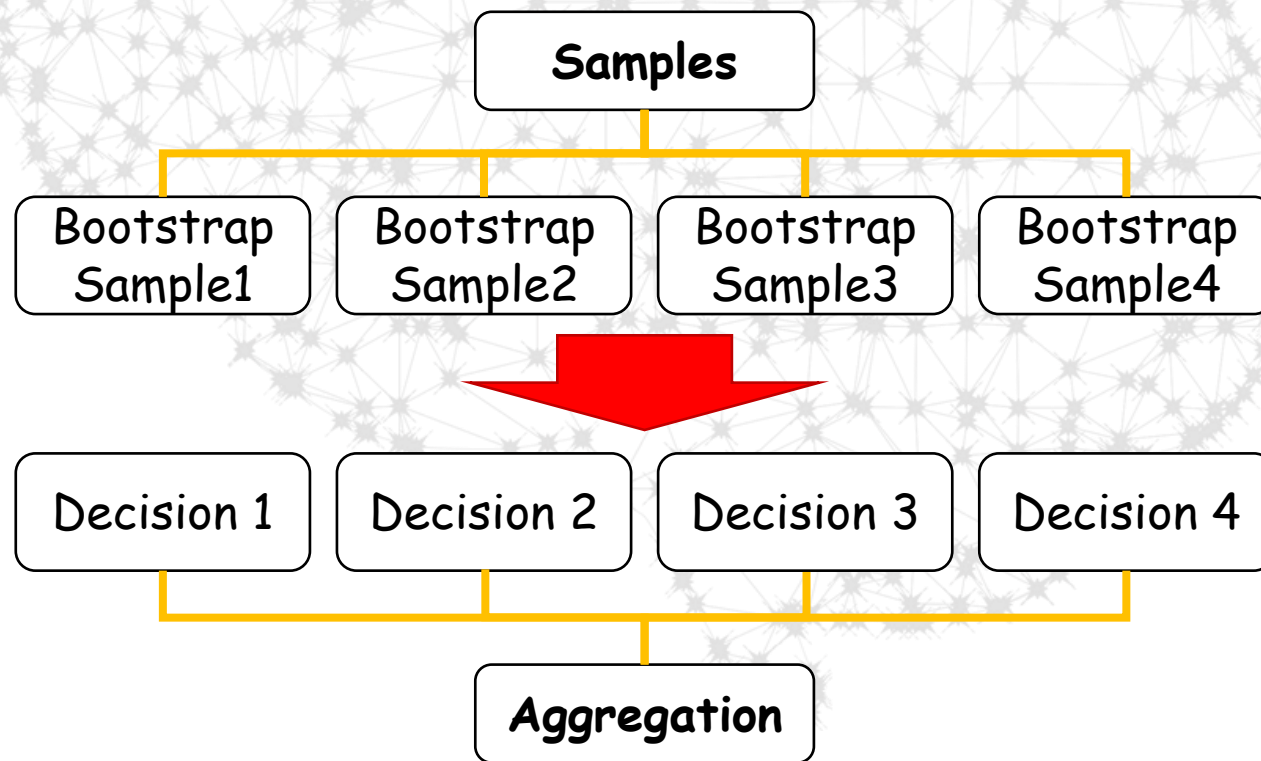
Random Forest

- Random forest model 안에는 `estimators_` 라는 속성이 있다.
- 그 속성 안에는 forest 를 이루는 각 tree 의 model 이 담겨져 있다.



Bagging

- Bagging 은 Bootstrap Aggregation 의 약자이다.
- Sample 데이터를 동일한 여러 모델(Bootstrap) 에게 전달하여
- 그 결과물들을 집계(Aggregation) 하여 **다수결 형식**으로 최종 결과를 도출 한다.

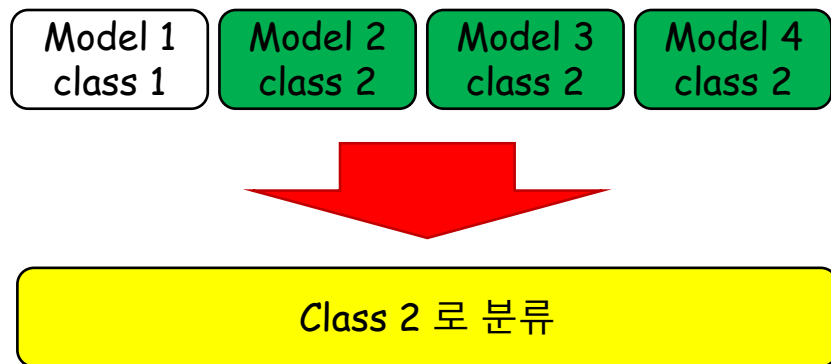


Voting

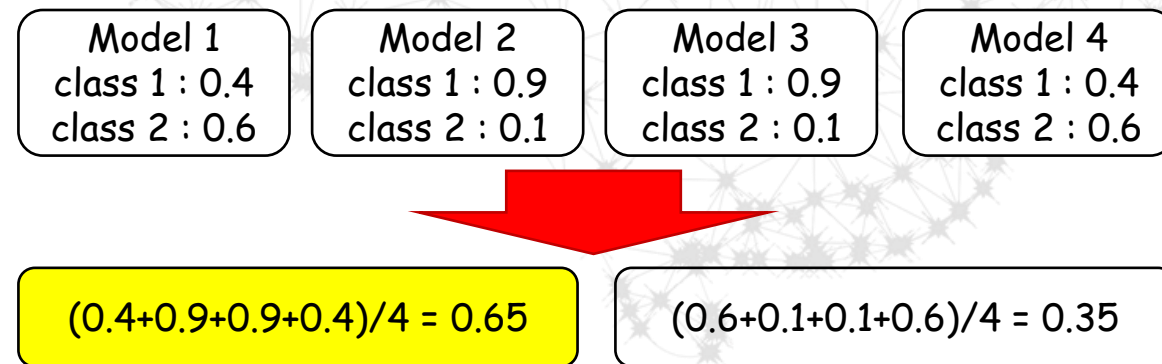
- 다수결 방식은 단순히 여러 모델을 조합하여 결과를 추출 후 다수결에 따라 선택하는 방식이다.
- **Bagging** 도 집계에 따라 다수결을 따르지만 **동일 모델**에 다른 데이터를 기준으로 한다면
- **Voting** 은 **서로 다른 모델** 들과 결합 할 수 있는 차이가 있다.
- 다수결 방식은 hard voting 과 soft voting 이 있다.

* 4개의 모델이 1, 2 class 를 분류하는 경우...

Hard Voting



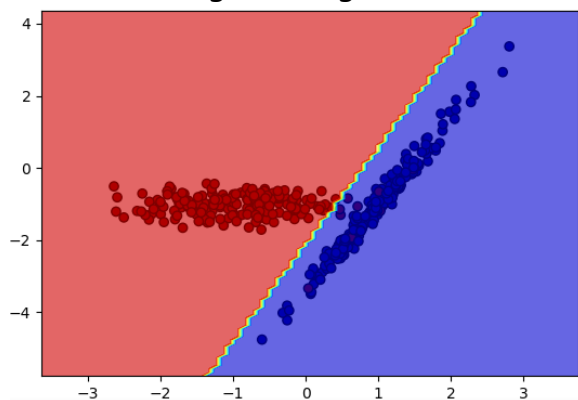
Soft Voting



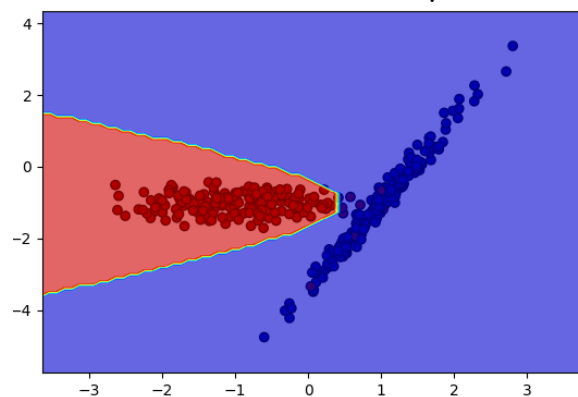
Voting

- 각 model 과 model 들의 다수결로 생성된 데이터를 비교해 보자
- 다른 모델들의 평균값을 받아들이는 모습이다.

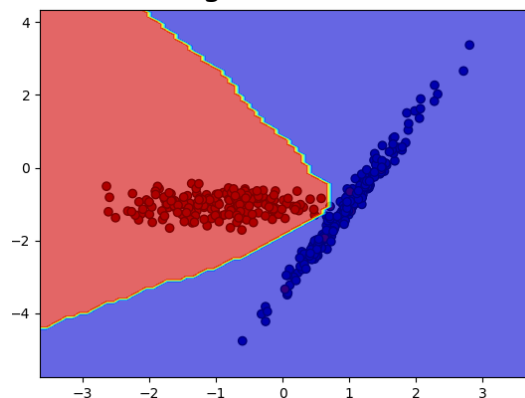
Logistic Regression



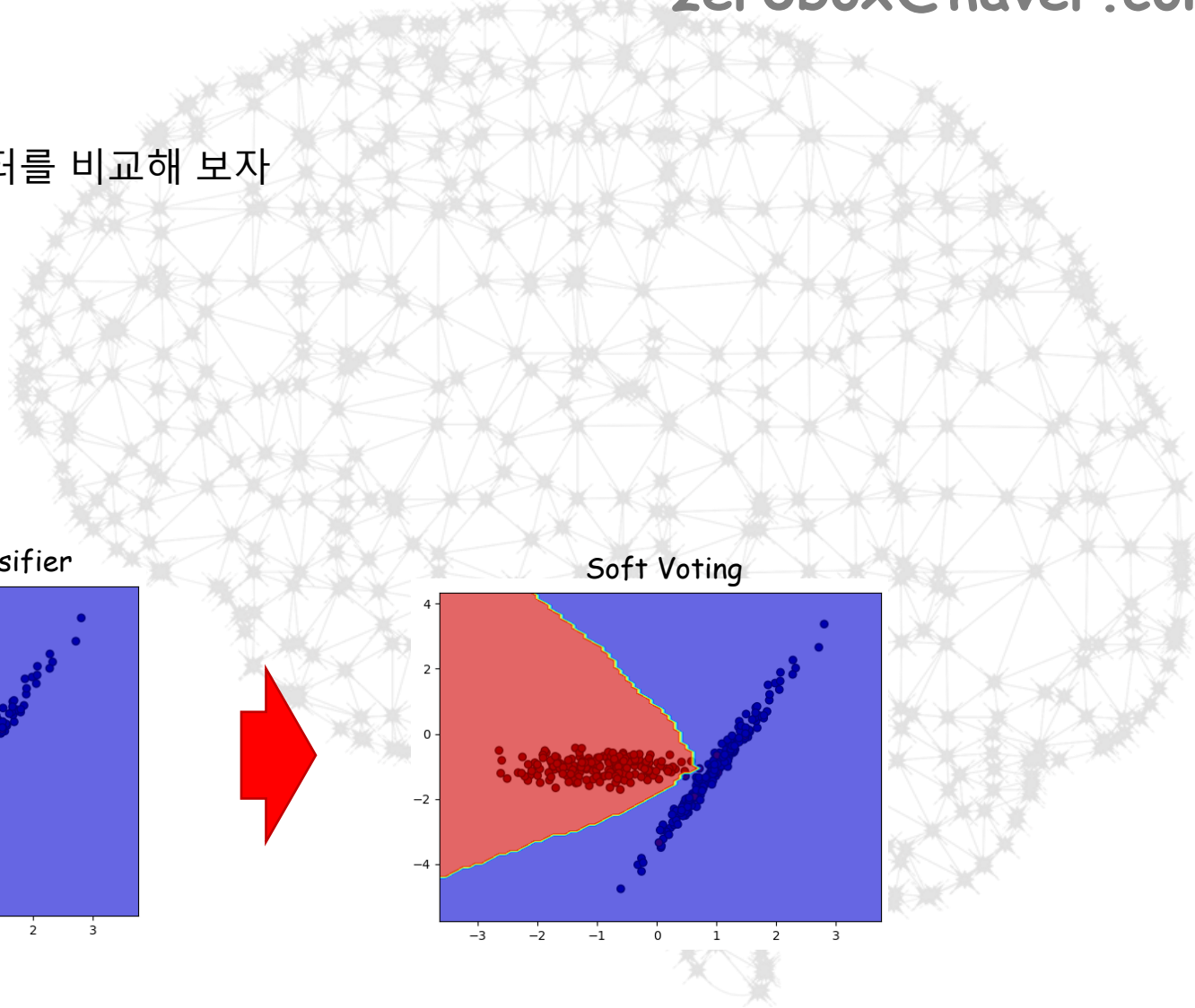
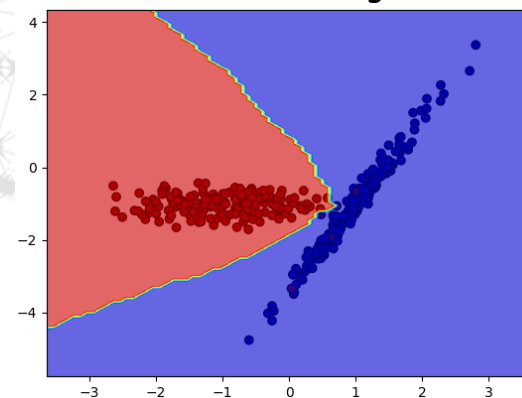
Gaussian Naïve Bayes



K-Neighbors Classifier

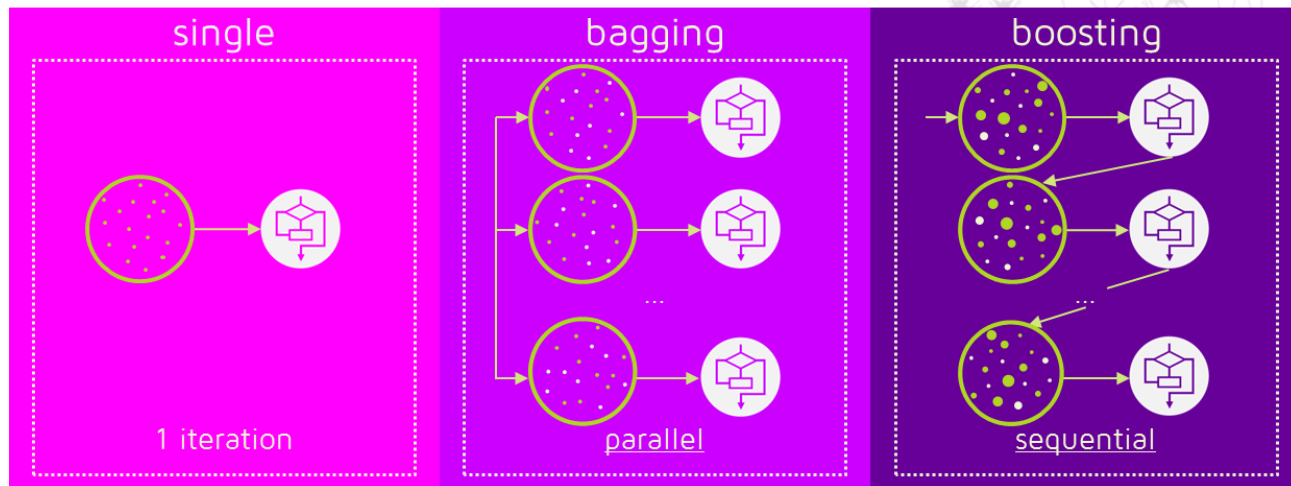


Soft Voting



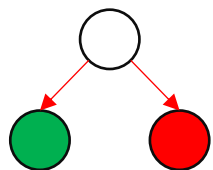
Boosting

- Boosting 은 하나의 모델로 시작해 하나씩 모델을 추가해 나가는 것이 특징이다.
- 아래 그림처럼 bagging 은 동시에 여러 모델을 실행하여 결과를 낸다면
- Boosting 은 이전 모델의 오차를 보완하는 다음 모델을 실행하는 방식을 택한다.

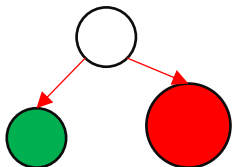


ADA Boost

- ADA 는 Adaptive(적응) 의 앞 글자를 따 왔다.
- 우선 약한 분류기(Weak classifier)를 이용하여 순차적으로 적용하는 방식을 확인 해 보자



- 약한 분류기(Weak classifier) : depth 가 낮은 tree, 이를 stump 라고 부른다.)



- 이전 tree 에서 잘못 분류한 샘플은 가중치를 높인다.



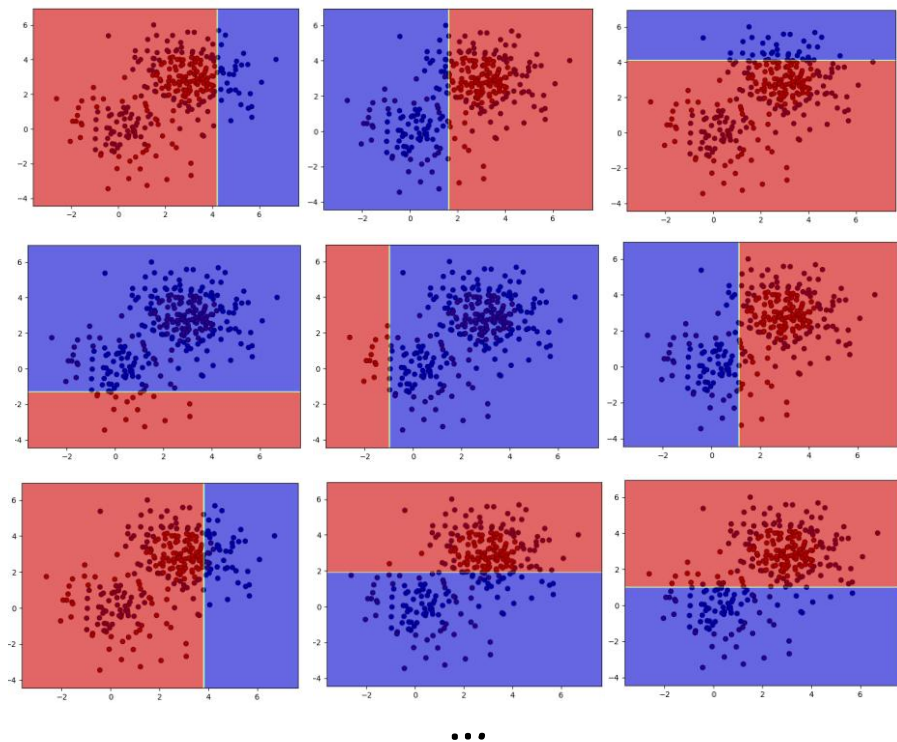
...

- 이런 식으로 단계가 계속 될수록 오차를 보완해 나가며 강한(Strong) 분류기 를 만들어 간다.

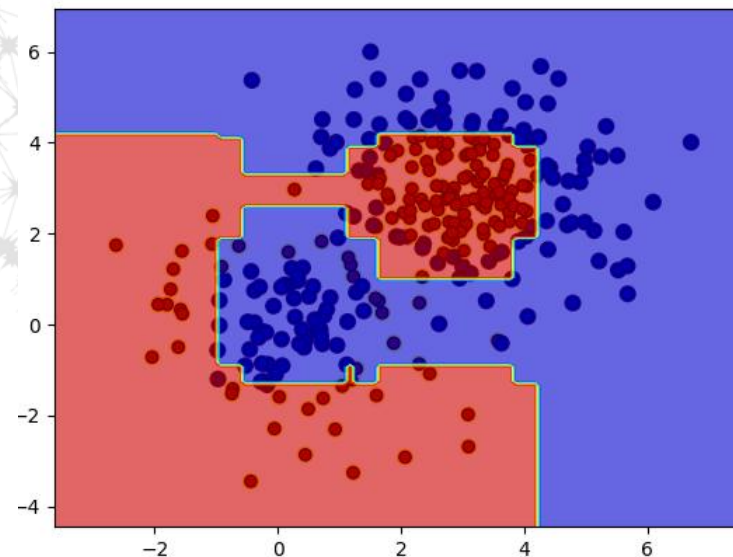
* 기출 문제를 풀고 **틀린 문제를 중심으로 공부**해 나가는 방식과 비슷하다고 볼 수 있다.

ADA Boost

- ADA 는 Adaptive(적응) 의 앞 글자를 따 왔다.
- 우선 약한 분류기(Weak classifier)를 이용하여 순차적으로 적용하는 방식을 확인 해 보자



실습파일 : 27_ada_boost.ipynb



Gradient Boost

- Gradient Boost 도 ADA Boost 처럼 이전 오차를 보정하면서 학습해 나간다.
- 다만 ADA 는 오차 샘플에 대해 가중치를 조절한다면
- Gradient 는 예측 치와 정답의 오차(residual error) 를 다음 학습에 활용해 나간다.

Seq.	Point	class
1	1	0
2	22	2
3	33	3
4	44	4
5	44	5
6	26	2

1~6 까지의 오차 차이를 계산하여 다음 학습에 반영 시킨다.

- Gradient Boost는 오차 차이(잔차)를 보완해 나가며 학습하기 때문에 과 적합의 문제가 있을 수 있다.
- 이를 보완하기 위한 여러 Boost 가 사용 된다.

실습파일 : 28_gradient.ipynb

요약(Summary)

1. 앙상블은 알고리즘의 성능개선을 위해 알고리즘들을 결합하는 것이다.
2. 앙상블은 Aggregation 방식과 Boost 방식이 있다.



Boost Library

Gradient Boost 기반의 Library

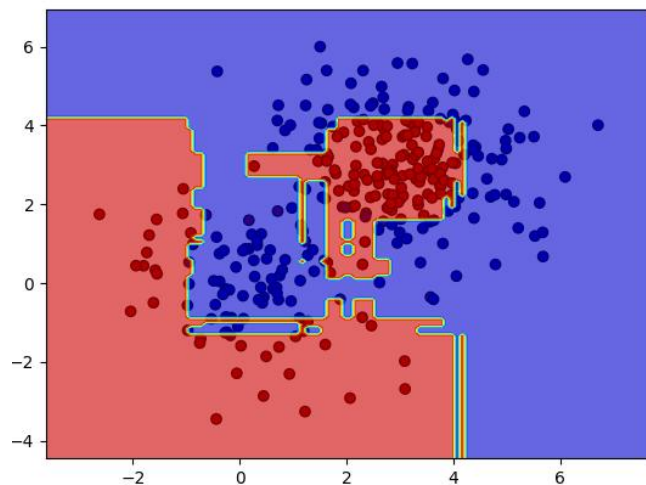
- 앞에 언급한대로 *Gradient Boost* 를 보완한 여러 라이브러리들이 있다.
- 대표 적으로 *XGBoost*, *LightGBM*, *CatBoost* 가 있다.
- *Kaggle* 등의 데이터 경진대회에 상위권을 차지하는 알고리즘 들이다.

Boost	Release	description
XG Boost	2014	GBM 을 발전 시킨 모델
Light GBM	2016	XG Boost 의 학습시간을 개선한 모델
Cat Boost	2017	GBM 을 계승하여 속도와 잔차 계산 방법에 대한 개선이 이루어진 모델

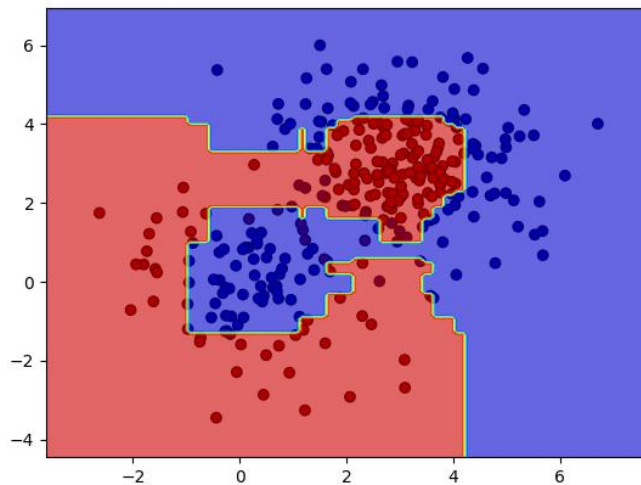
XG(eXtra Gradient) Boost

- XG 는 병렬(parallelization)처리로 기존(GBM) 보다 속도가 향상 되었다.
- 가지치기, 조기 중단 등을 통해서 overfitting 방지 에 대해서도 개선 되었다.

Gradient Boost



XG Boost

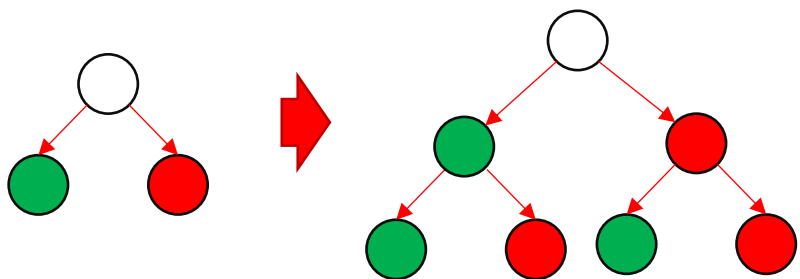


실습파일 : 29_xg_boost.ipynb

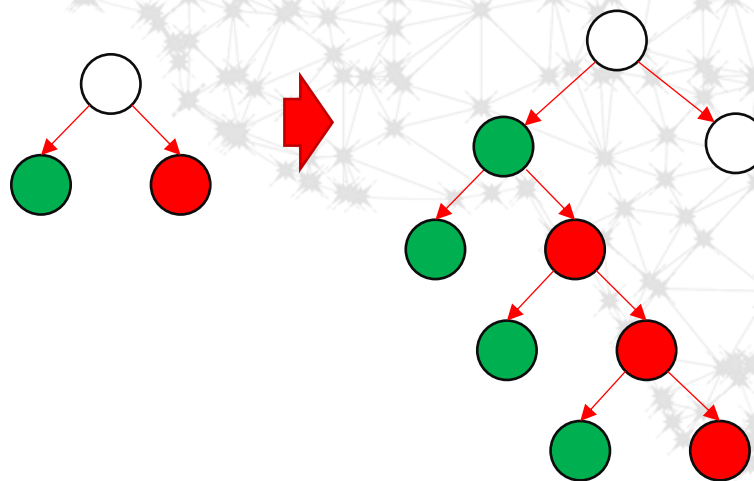
Light GBM(Gradient Boost Model)

- XG 보다 학습 속도가 더 개선 되었다.
- 다만 적은 수(10,000건 미만)의 데이터를 사용하면 **overfitting** 우려가 있다.
- Light GBM 은 Tree 생성시 균형트리분할(Level Wise) 가 아닌 리프 중심 분할(Leaf Wise) 를 채택했다.
- 때문에 속도는 빠르지만 과 적합에 대한 문제가 있을 수 있다.

GBM & XG Boost



Light GBM



실습파일 : 30_light_gbm.ipynb

4. Cat(Categorical) Boost

- Cat 은 Level Wise Tree 로 tree 를 만든다.(light GBM 에서만 leaf-wise 사용)
- 또한 GBM 에서 사용한 오차 차이(잔차) 계산 방법이 달라졌다.(Ordered Boosting)
- 또한 데이터 전처리 시 범주형 변수를 활용하여 크기를 줄인다.(Categorical Feature)

Ordered Boosting

Seq.	Point	class
①	1	0
②	22	2
3	33	3
4	44	4
5	44	5
6	26	2

1의 잔 차를 계산해 모델을 만들어 2를 예측

1, 2의 잔 차를 계산해 모델을 만들어 3을 예측

Categorical Feature

country	Hair color	class
Korea	Black	1
Korea	Black	1
Russia	Gold	0
Russia	gold	0
하나의 feature 로 취급		

- 결측치가 많은 데이터(sparse matrix)에 대해서는 부적합 하다
- 수치형 변수가 많은 경우 light BGM 보다 학습 시간이 오래 걸린다.

요약(Summary)

1. GBM 알고리즘의 개선을 위한 외부 라이브러리 알고리즘들이 있다.
2. XG Boost, light GBM, Cat Boost 이다.
3. XG -> light -> cat 순으로 나왔지만 저마다의 장단점이 있다.



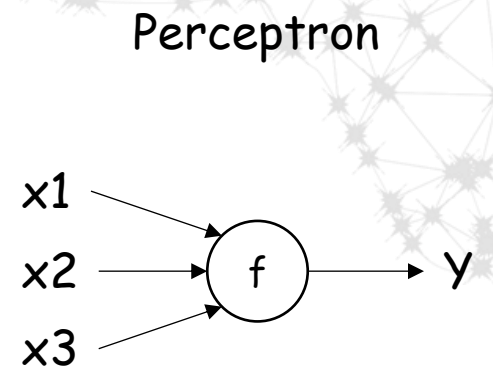
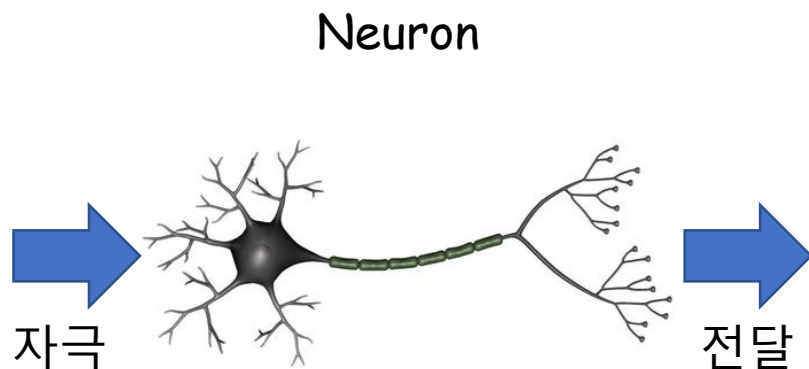
Deep Learning

Deep Learning

- Deep learning 은 tensor flow, keras, py-torch 가 특화되어있다.
- Scikit learn 에서도 Deep learning 의 일부인 신경망(neural network)을 다룰 수 있다.
- 비정형(이미지, 음성) 데이터 에서는 신경망이 압도적인 성능을 보인다.

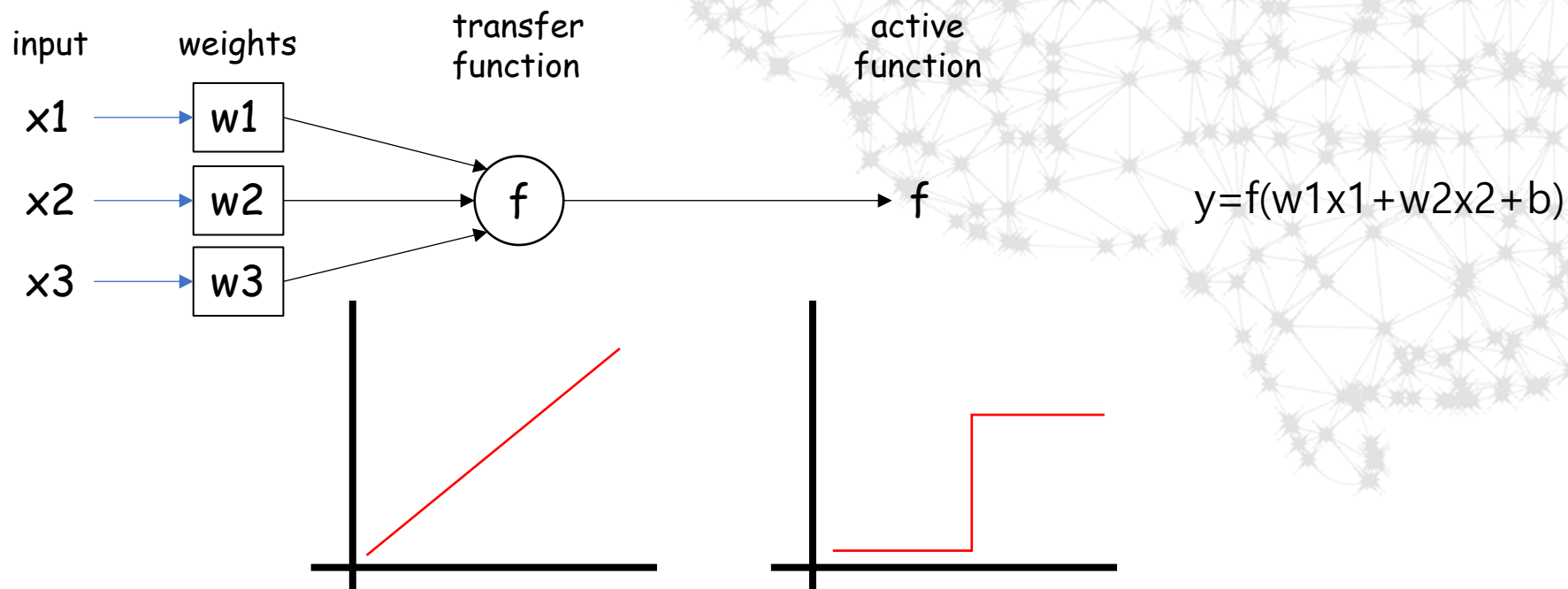
Perceptron

- 인공 신경망은 실제 뉴런이 여러 자극을 받아서 전달하는 구조를 모방한 것이다.
- 뉴런처럼 여러 입력을 받아 하나의 출력을 내는 이 구조를 perceptron 이라고 부른다.



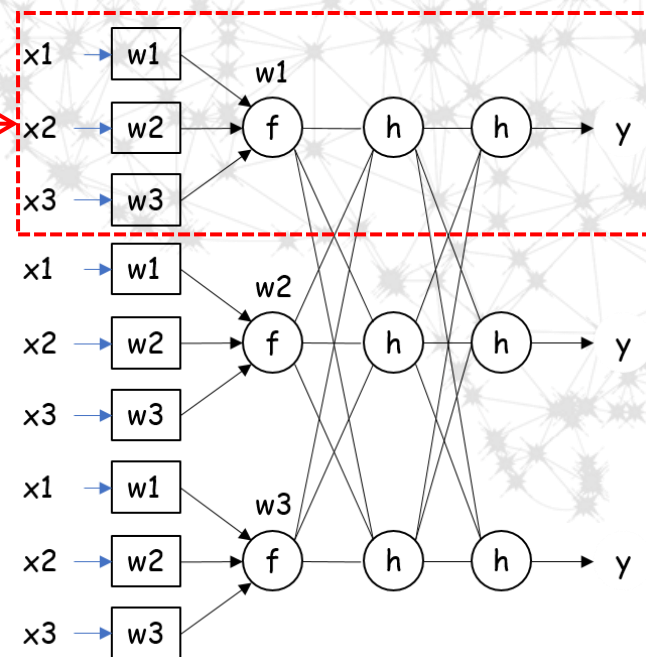
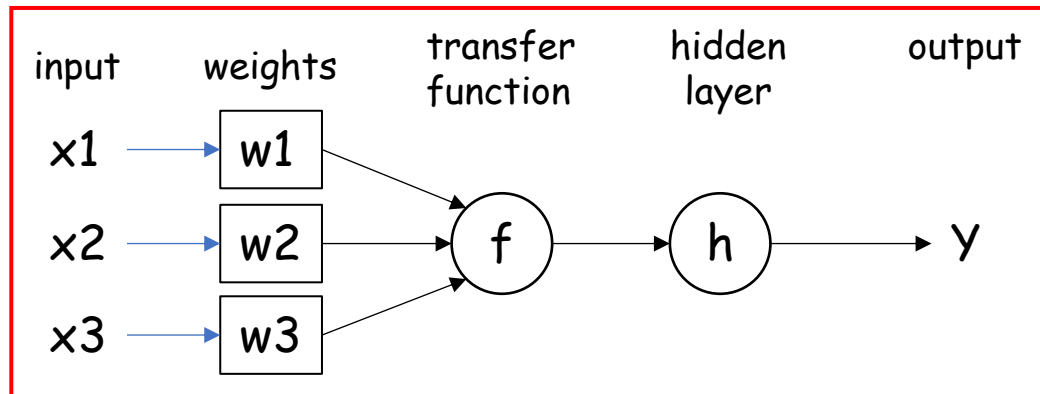
단층 Perceptron

- 각 입력에는 가중치(weight)가 존재한다.
- Transfer function 은 입력 값을 선형식으로 변경한다.
- Active function 을 거쳐 단순한 선 형태를 보다 의미 있는 분류를 위한 형태로 변경 한다.
- 아래와 같은 구조를 우리는 단층 perceptron 이라 부른다.
- 단층 perceptron 은 선형 구조 이기 때문에 XOR 문제에 취약 하다.



MLP(Multi Level Perceptron)

- 선형 구조의 문제를 해결하기 위해 단계(Layer)를 추가하여 좀더 복잡도를 높일 수 있다.
- 이때 중간 단계를(Layer) 은닉 층(hidden Layer) 라고 부른다.
- 각 active function 은 다시 입력 값이 되어 가중치를 부여 받게 된다.
- 이러한 단계가 여러 개가 될 수 있어 우리는 다중 perceptron 이라고 부른다.



MLP(Multi Level Perceptron)

- MLP 를 활용하여 분류를 해 보자

```
MLPClassifier(  
    solver='lbfgs',  
    random_state=0,  
    max_iter=100000,  
    hidden_layer_sizes=(3, 3, 3, 3)  
) .fit(x, y)
```

실습파일 : 32_mlp.ipynb

경사 하강 알고리즘 선택(39페이지)

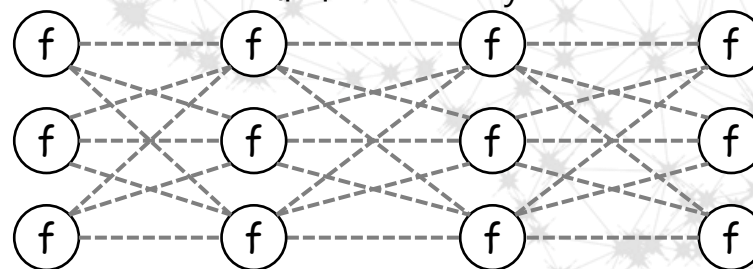
lbfgs : 속도가 빠르고 적은 데이터에 최적화

sgd - 데이터가 많은 경우, 가중치를 점진적으로 갱신, 속도가 느림

adam(기본) : 사용이 쉽고 대부분의 데이터에서 안정적인 속도와 성능

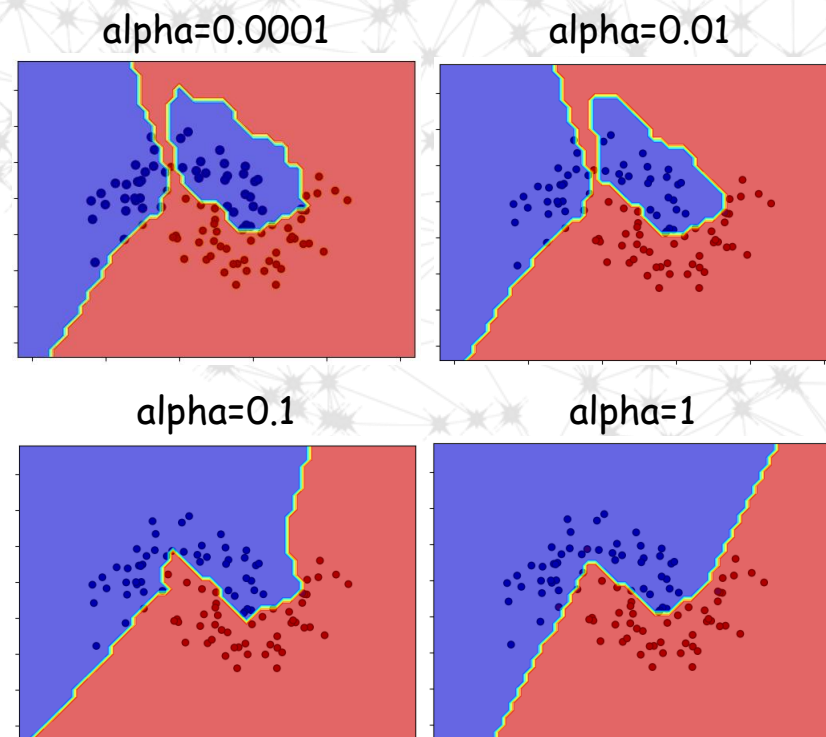
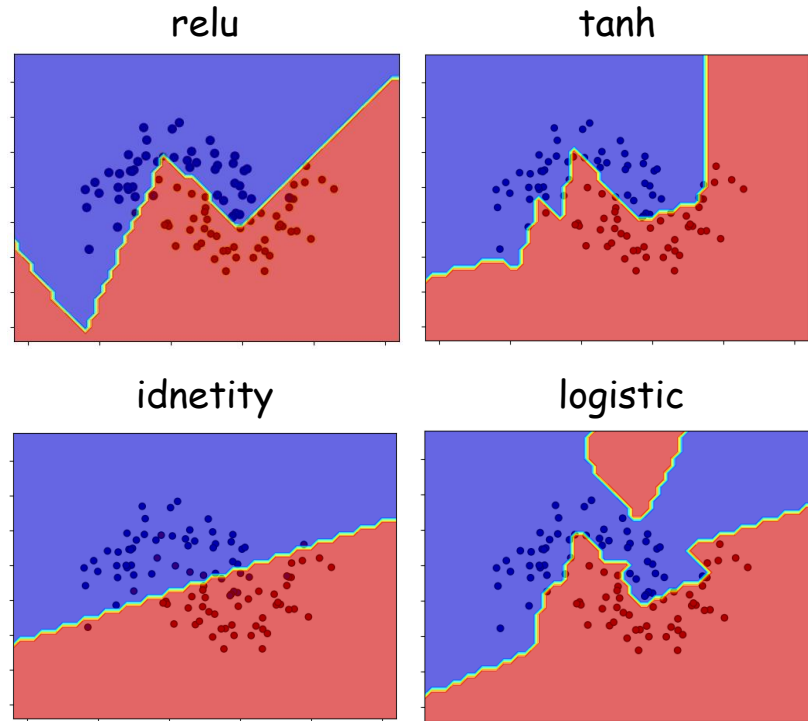
경사 하강 알고리즘 에서 최대 반복 횟수

3 개의 perceptron 으로 이루어진
4개의 hidden layer



Parameter option

- Hidden layer 에 따라서 결정 경계의 복잡도가 증가하는 것을 보았다.
- Activation 옵션에 따라서 같은 은닉층의 같은 퍼셉트론을 사용해도 다른 결정 경계가 나타난다.
- Alpha 는 제약 강도를 의미한다.

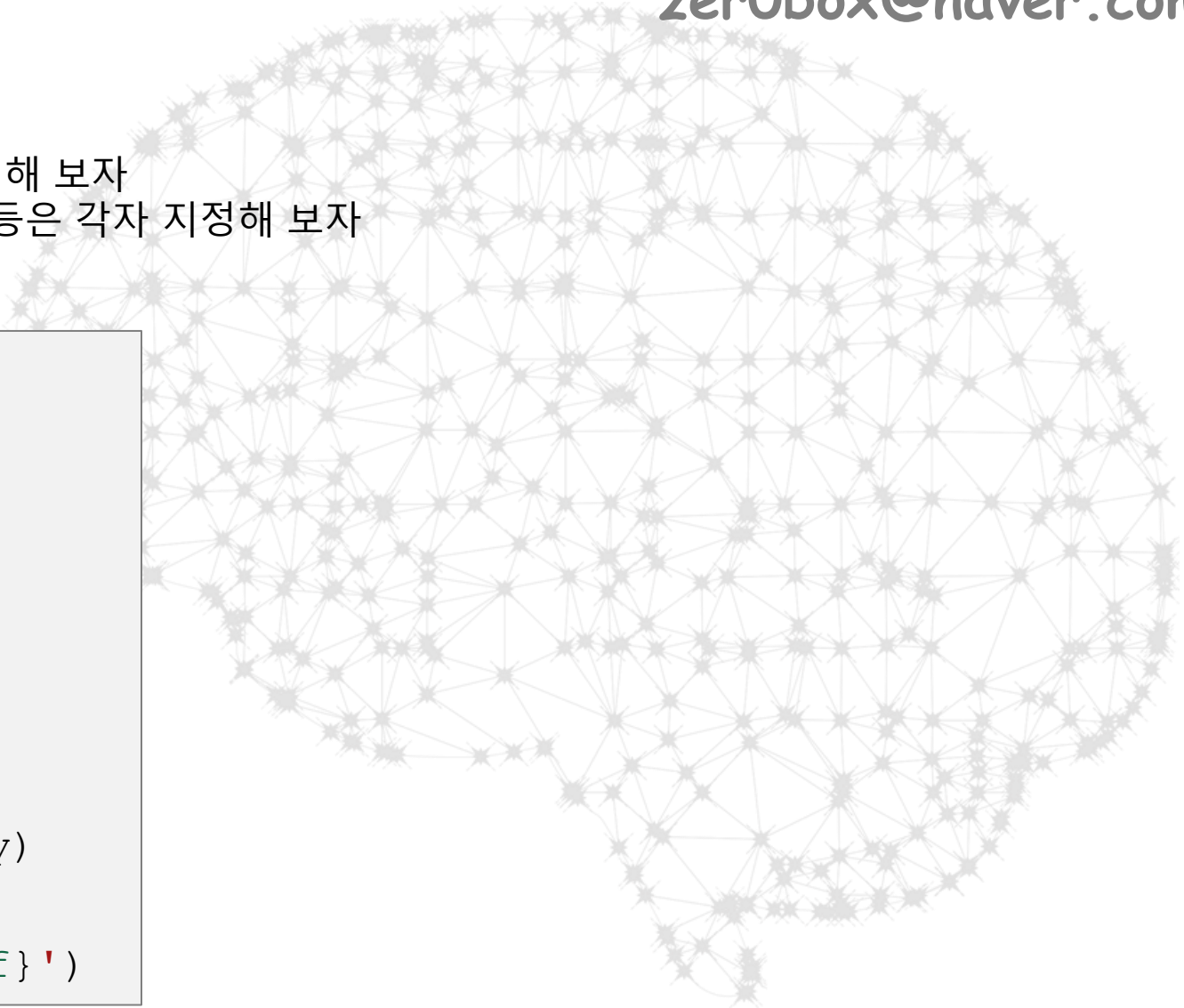


MLP 활용

- MLP 분류 알고리즘을 활용하여 iris 분류를 진행해 보자
- solver, hidden_layer_sizes, alpha, activation 등은 각자 지정해 보자

```
model = nn.MLPClassifier(  
    solver='adam',  
    max_iter=5000,  
    hidden_layer_sizes=(50,),  
    random_state=33,  
    alpha=0.001,  
    activation='relu')  
  
score = cross_val_score(model,x,y)  
  
print(f'score : {score.mean():.2f}')
```

실습파일 : 34_iris_test.ipynb



요약(Summary)

인공 신경망 의 장점

- 대량의 데이터에서 정교하고 복잡한 모델을 만들어 낼 수 있다.
- 충분한 연산 시간과 세심한 파라미터 조정을 통해 성능을 극한으로 끌어 낼 수 있다.

인공 신경망 의 단점

- 크고 강력한 모델 일 수록 학습 시간이 다른 알고리즘에 비해 오래 걸린다.
- SVM 과 마찬가지로 scale 이 필요하다.
- 매개변수의 수가 굉장히 많고(우리가 배운 건 아주 일부분) 세심한 조정이 필요하다.

요약(Summary)

1. 완벽한 인공지능 알고리즘은 없다.
2. 각 알고리즘의 장단점을 파악해 적합한 알고리즘을 선택해야 한다.